
PROCEDURAL QUEST GENERATION FOR ROLE-PLAYING GAMES USING LARGE LANGUAGE MODELS

B.TECH PROJECT REPORT

*submitted towards the partial fulfilment of the
requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

SOFTWARE ENGINEERING

Submitted by

TASMIYA

2K21/SE/181

tasmiya_se21b1_044@dtu.ac.in

YASHVARDHAN SINGH

2K21/SE/196

yashvardhansingh_se21a15_57@dtu.ac.in

Under the supervision of

DR. DIVYASHIKHA SETHIA

Associate Professor



DEPARTMENT OF SOFTWARE ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY

CERTIFICATE

This is to certify that the project report titled "Procedural Quest Generation for Role-Playing Games Using Large Language Models" submitted by Tasmiya (2K21/SE/181) and Yashvardhan Singh (2K21/SE/196) for the partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Software Engineering is a record of the project work carried out by the students under my supervision. To the best of my knowledge, this work has not been submitted in part or full for any degree or diploma to this university or elsewhere.



Place: Delhi

Date: May 20, 2025

DR. DIVYASHIKHA SETHIA
SUPERVISOR

Department of Software Engineering
Delhi Technological University

DECLARATION

I hereby declare that the project report work entitled "Procedural Quest Generation for Role-Playing Games Using Large Language Models" which is being submitted to Delhi Technological University in partial fulfilment of requirements for the award of the degree of Bachelor of Technology in Software Engineering is a bonafide report of B.Tech Project carried out by us. The material contained in the report has not been submitted to any university or institution for the award of any degree or diploma.



Place: Delhi

Date: May 20, 2025

TASMIYA

2K21/SE/181

Department of Software Engineering
Delhi Technological University



Place: Delhi

Date: May 20, 2025

YASHVARDHAN SINGH

2K21/SE/196

Department of Software Engineering
Delhi Technological University

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude to all those who played a significant role in the successful completion of this project. This work would not have been possible without the contributions, support, and encouragement of many individuals, each of whom has been instrumental in guiding us throughout this journey.

First and foremost, we are profoundly grateful to Dr. Divyashikha Sethia, our esteemed project supervisor, for her unwavering support, insightful guidance, and constructive feedback. Her academic expertise, patience, and dedication provided the foundation for our research, and her encouragement helped us stay focused and motivated during challenging phases of the project. Her commitment to excellence consistently inspired us to elevate the quality of our work.

We are also sincerely thankful to the faculty members and administrative staff of the Department of Software Engineering. Their assistance, whether through academic advice, logistical support, or providing access to departmental resources, has been crucial in enabling the smooth execution of various stages of this project.

Additionally, we extend our sincere thanks to our families and close friends for their patience, encouragement, and understanding. Their belief in our potential and emotional support served as a constant source of strength throughout the duration of this project. They provided the stability and reassurance we needed to persist and perform to the best of our abilities.

In conclusion, we express our deepest appreciation to everyone who contributed in any way to the realization of this work. Every suggestion, encouragement, and act of support—no matter how small—played a vital part in the completion of this project.

This project stands as a testament to the shared efforts of the following contributors:

Tasmiya (2K21/SE/181), and **Yashvardhan Singh** (2K21/SE/196).

ABSTRACT

This project investigates the use of large language models (LLMs) for procedural quest generation (PQG) in role-playing games (RPGs) to enhance narrative diversity and reduce manual content authoring. Procedural content generation (PCG) has long been employed in game design to create expansive and dynamic game worlds, but narrative elements—particularly quests—remain challenging to automate due to their reliance on coherence, structure, and context. To address this, we utilize modern transformer-based LLMs capable of producing contextually rich text based on structured input representations. The dataset used in this project is derived from publicly available quest data, formatted into a consistent XML-inspired schema with clearly defined tags representing quest components such as objectives, tasks, characters, rewards, and locations. This structured format allows the models to focus on learning narrative patterns while preserving semantic alignment with the game world. To enable efficient training on limited hardware, we adopt a two-stage fine-tuning approach. First, the models—GPT-2, LLaMA 3.2, and TinyLlama—are quantized using 4-bit techniques to reduce memory consumption. Then, parameter-efficient fine-tuning (PEFT) is performed using Low-Rank Adaptation (LoRA), which injects trainable adapters into selected model layers, significantly lowering the number of trainable parameters while preserving performance. A custom pipeline is developed to manage the tokenization, quantization, and LoRA integration, enabling fine-tuning on constrained setups. The quality of generated quests is evaluated using both automatic metrics—BLEU, METEOR, BERTScore, and perplexity—and manual inspection for narrative consistency and fluency. Results show that larger LLaMA models generally outperform GPT-2 variants in producing structured and coherent quests, although trade-offs exist in training time and resource usage. This project demonstrates that with minimal supervision and efficient adaptation techniques, LLMs can effectively generate believable and diverse RPG quests. The findings lay the groundwork for scalable narrative generation systems that could be integrated into modern game engines and development pipelines.

LIST OF FIGURES

1.1	Radial diagram showing key thematic research gaps in procedural quest generation with LLMs, highlighting challenges in narrative control, datasets, coherence, evaluation, integration, scalability, and personalization.	3
2.1	Adapter module integration by Houlsby et al. [36]	12
2.2	Transformer architecture by Vaswani et al. [5]	15
2.3	Transformer and autoregressive generation using tokenized inputs and a softmax output layer by Radford et al. [37]	16
2.4	Circular flow diagram illustrating the core components of a quest in game design: actors, objectives, constraints, and rewards. Each component plays a critical role in shaping player experience and quest structure.	18
4.1	Flow diagram of the procedural quest generation system. It illustrates the modular pipeline used for model training and evaluation.	25
4.2	Cover arts for selected RPGs used in the QUEST dataset [10, 13, 15] . . .	28
5.1	Training diagnostics for fine-tuned models. The plots display loss progression, gradient norm stability, and overall training time, offering insight into optimization dynamics and efficiency.	37
5.2	Automated evaluation results across models. The plots illustrate language modeling quality (Perplexity) and output similarity with reference quests using BLEU, METEOR, and BERTScore.	38

LIST OF TABLES

1.1	Scope of the PQG system, including key features and constraints	5
1.2	Summary of key limitations observed in the proposed PQG approach . . .	5
2.1	Comparison of GPT-2 model variants in terms of parameters	10
2.2	Comparison of GPT-2 model variants with LLaMA-3.2-1B-Instruct in terms of size, architecture, and instruction-following ability	11
2.3	Overview of common PEFT techniques used for model adaptation	12
2.4	Common quantization methods used to compress and accelerate LLMs . .	14
2.5	Key benefits of applying quantization to language models	14
2.6	Analysis of trade-offs introduced by model quantization	15
2.7	Comparative analysis of common PQG methods	19
3.1	Summary of works focused on structured quest generation techniques . . .	21
4.1	Distribution of collected quest data categorized by the game title	28
4.2	Quantization configurations used in model compression for training	30
4.3	Configuration of LoRA adapter modules integrated during fine-tuning . . .	30
4.4	Features and components included in the training pipeline	31
4.5	Settings used for text generation tasks	32
4.6	Qualitative criteria used for evaluating generated quests	34
5.1	Training and evaluation metrics reported for each fine-tuned model	36
5.2	Detailed quantitative evaluation results for each fine-tuned model	36
5.3	Comparison of quest generation strategies and outcomes between Värtinen et al. [24] and this study.	39
A.1	Field description in the JSON schema proposed by Värtinen et al. [24] . .	50
B.1	Hardware specifications used for training and evaluation	53
B.2	Software libraries, versions, and tools used to implement the PQG system .	53
B.3	List of pre-trained models used throughout the experimental setup	54
B.4	Data preprocessing steps applied to the raw quest data before training . .	54
B.5	Hyperparameter configurations used during the fine-tuning phase	55
B.6	Hyperparameter settings employed during the text generation phase	55
B.7	List of automated evaluation metrics and their respective roles	56

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Background	2
1.2 Research Gaps	3
1.3 Proposed Solution	5
1.4 Problem Statement	6
1.5 Objectives & Contributions	7
1.6 Rest of the Report	7
2 Technical Background	9
2.1 Large Language Models (LLMs)	9
2.1.1 GPT-2	10
2.1.2 LLaMa 3.2	10
2.1.3 TinyLlama	11
2.2 Parameter-Efficient Fine-Tuning (PEFT)	11
2.2.1 LoRA: Low-Rank Adaptation of LLMs	12
2.3 Model Quantization	13
2.3.1 Quantization Techniques (e.g., 4-bit, 8-bit)	14
2.3.2 Benefits and Trade-offs	14
2.4 Transformers and Text Generation	15
2.4.1 Causal Language Modeling	16
2.4.2 Tokenization and Prompt Formats	17
2.5 Procedural Quest Generation	17
2.5.1 Structure and Components of a Quest	17
2.5.2 Procedural Generation in Quest Design	18
3 Related Work	20
3.1 Procedural Content Generation in Games	20
3.2 NLP Techniques for Quest Generation	21
3.3 Human Evaluation of Generated Content	22
4 Methodology	24
4.1 Workflow for Procedural Quest Generation	24
4.1.1 Dataset Collection and Formatting	26
4.1.2 Model Selection and Evaluation	26
4.1.3 Framework Scope and Limitations	27
4.2 Dataset Description and Preprocessing	27
4.2.1 Source and Specification of Dataset	27

4.2.2	Tagging and Formatting	29
4.3	Model Configuration and Training	29
4.3.1	Quantization Workflow	30
4.3.2	LoRA Adapter Integration	30
4.3.3	Training Pipeline Setup	31
4.3.4	Text Generation Settings	31
4.4	Evaluation Strategy	32
4.4.1	Perplexity	32
4.4.2	BLEU	32
4.4.3	METEOR	33
4.4.4	BERTScore	33
4.4.5	Qualitative Analysis	33
5	Results and Discussion	35
5.1	Training Results	35
5.2	Quantitative Evaluation Results	36
5.3	Discussion	38
6	Conclusion and Future Work	41
6.1	Summary of Contributions	41
6.2	Limitations of the Study	42
6.3	Suggestions for Future Work	42
6.4	Conclusion	44
A	Format for the Quest Dataset	49
B	Experimental Setup and Hyperparameters	52

Chapter 1

Introduction

In this chapter, we present an overview of the foundational concepts, motivations, and methodological approaches underlying the development of a procedural quest generation (PQG) system for role-playing games (RPGs) using large language models (LLMs). The field of procedural content generation (PCG) has been a long-standing area of interest in game design, offering a scalable and adaptive means of creating game elements such as maps, levels, and narratives.

The process of PCG involves algorithmically creating game content—such as environments, items, characters, and narratives—through predefined computational methods, rather than relying solely on manual authoring by human designers. The central aim of PCG is to automate aspects of game content creation in a way that reduces development time and labor while increasing variability, replayability, and scalability.

Procedural generation techniques are typically rule-based or stochastic in nature, incorporating elements of randomness, heuristics, grammar systems, or machine learning to ensure content diversity and structural coherence. Depending on the design goals, PCG systems can be fully autonomous or require designer-guided parameters to maintain thematic and gameplay consistency. These systems are particularly useful in large-scale or open-world games, where the manual creation of unique assets for every player experience is impractical.

The history of PCG spans both digital and analog gaming traditions. Early digital games such as *Rogue* (1980) [1] pioneered procedurally generated dungeons, laying the foundation for the “roguelike” genre and demonstrating how algorithmic systems can create replayable gameplay experiences. On the analog side, table-top game systems like *Dungeons & Dragons* [2] introduced rule-driven generation of encounters and storylines, influencing digital design philosophies. Modern examples such as *Minecraft* [3] employ PCG to generate endless, explorable terrain, while *AI Dungeon* [4] uses language models to dynamically create interactive narrative experiences. These systems highlight the versatility of PCG in generating spatial, mechanical, and narrative content across genres.

Game content generated through procedural methods covers a broad spectrum, including spatial elements like terrain, maps, and dungeons (as seen in *Rogue* [1] and *Minecraft* [3]), mechanical components such as enemy behaviors and item properties, and narrative structures involving dialogue trees, quests, and branching storylines. Early PCG systems often produced repetitive or structurally simple outputs focused primarily on level or map generation. However, recent advances in computational techniques—particularly the integration of artificial intelligence (AI) and natural language processing (NLP)—have greatly expanded the expressive power of PCG. Modern methods now enable the gener-

ation of complex, human-like storytelling and adaptive game experiences that respond dynamically to player interactions, thereby enhancing immersion and replayability.

However, quest generation, in general, presents a unique challenge due to the structured yet narrative-rich nature of quests that must align with both gameplay mechanics and storytelling goals.

Traditional approaches to quest generation have relied heavily on handcrafted templates, which often lack the linguistic fluency and variability required for immersive player experiences. Recent advances in NLP, particularly with transformer-based architectures such as GPT and LLaMA, have demonstrated exceptional capabilities in generative tasks, including dialogue modeling, summarization, and storytelling [5, 6]. These models are pre-trained on large corpora and can be fine-tuned with domain-specific datasets to generate coherent and context-aware narrative content.

To address the specific task of quest generation, this project proposes a hybrid methodology combining quantized language models with parameter-efficient fine-tuning (PEFT) via LoRA adapters [7–9]. By leveraging dialogue and quest corpora extracted from well-established RPGs such as the *Fallout* series [10, 11], the *Baldur’s Gate* series [12, 13], and the *Elder Scrolls* series [14, 15], the system aims to generate semantically valid, context-sensitive quests using a structured prompt format. The resulting system is designed to be both memory-efficient and scalable for integration in real-time or offline game pipelines.

1.1 Background

RPGs have long relied on meticulously authored content to deliver immersive narratives and interactive experiences. Traditionally, the creation of such content—especially quests and dialogue—has required extensive manual effort from writers and designers. This manual process limits scalability and often constrains replayability. To address these issues, PCG techniques have been explored to automate the creation of game elements. While PCG has been successfully used for generating maps, characters, terrains, and items, narrative generation remains a relatively underdeveloped area due to its dependence on linguistic coherence, contextual consistency, and player agency [16].

The integration of natural language generation (NLG) into PCG pipelines has opened new possibilities for automating quest creation that balances narrative richness with structural logic. Early procedural narrative systems often relied on handcrafted templates or symbolic AI techniques, such as story grammars [17] and planning-based approaches [18]. Some systems also model quests as directed graphs or trees, encoding dependencies and branching logic to support replayable and nonlinear narratives [19]. However, these methods struggled to produce diverse, semantically rich, or context-aware quests, often resulting in repetitive or mechanically constrained narratives.

Recent advances in large-scale transformer-based language models—such as GPT [6] and LLaMA [20]—have significantly altered this landscape. These models, pre-trained on massive text corpora, are capable of generating fluent, coherent, and contextually relevant textual outputs that approximate human writing styles across diverse domains. This enables the procedural generation of branching quests, dialogue trees, and interactive storytelling sequences with greater variability and linguistic naturalness. For example, *AI Dungeon* [4] leverages GPT-based models to create open-ended, player-driven narratives that adapt dynamically to input prompts.

To effectively apply these models in games, structured datasets that include annotated dialogue and quest sequences are essential. Extracted corpora from games such as

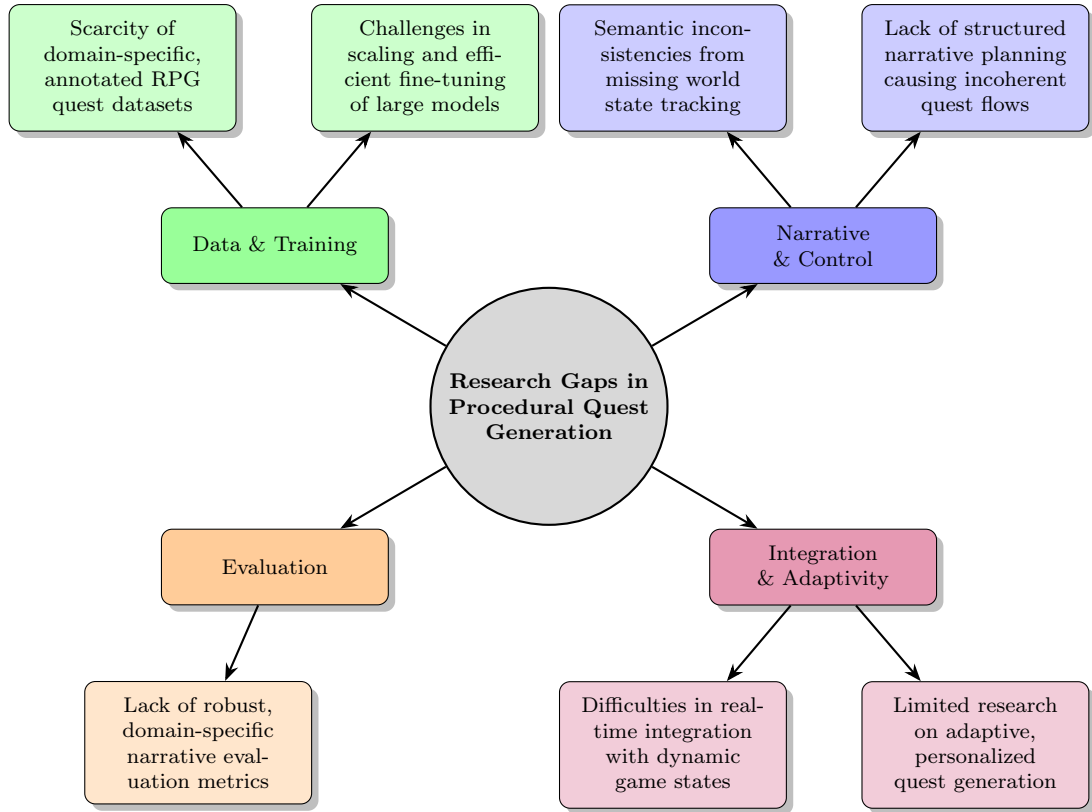


Figure 1.1: Radial diagram showing key thematic research gaps in procedural quest generation with LLMs, highlighting challenges in narrative control, datasets, coherence, evaluation, integration, scalability, and personalization.

Fallout [10, 11], *Baldur’s Gate* [12, 13], and *The Elder Scrolls* [14, 15] provide valuable resources for training these systems. These datasets allow the models to learn not only the language used in quests but also their structure, tone, and logical flow.

Additionally, the ability to generate semantically relevant and structurally sound quests can significantly enhance dynamic storytelling and replayability, thus paving the way for adaptive and personalized game experiences.

1.2 Research Gaps

While the use of LLMs for PQG presents exciting possibilities, a number of research gaps hinder the development of reliable, scalable, and context-sensitive systems. These challenges are rooted in the intersection of natural language generation, narrative theory, and real-time game design, and must be addressed to advance the field toward more adaptive and immersive game experiences (see Figure 1.1).

One of the most fundamental limitations is the lack of structured narrative control in existing LLMs. Although models like GPT and LLaMA demonstrate strong surface-level fluency, they are inherently autoregressive and lack high-level narrative planning mechanisms [21]. This leads to inconsistencies in quest structure—such as missing subgoals, conflicting motivations, or incoherent resolution paths. Unlike manually authored quests, which follow well-defined narrative arcs (e.g., setup→complication→resolution), LLM outputs often lose track of long-range dependencies across quest stages. Some hybrid systems have introduced graph-based planning or symbolic scaffolds (e.g., plot graphs, quest

trees), but few have been effectively integrated with LLMs in a seamless and generalizable way [22, 23].

A second challenge involves the limited availability of domain-specific fine-tuning resources. While general-purpose language models are trained on massive, heterogeneous corpora, they lack detailed exposure to the linguistic patterns, structural tropes, and character archetypes specific to RPGs. The adopted hypothesis from related works is that the dataset used to pretrain GPT-2 does not have enough number of quest examples to generate high-quality quest results without further fine-tuning with specialized datasets [24, 25]. Fine-tuning on curated corpora from various games can help models learn quest-specific syntax and semantic flow. However, such datasets are rarely available in clean, annotated formats that are aligned with narrative objectives or quest semantics. The absence of open-source, standardized RPG quest datasets remains a major bottleneck for reproducible research.

Even after fine-tuning, certain LLMs often exhibit semantic inconsistencies and logical drift [26, 27]. These issues commonly manifest as contradictions between non-player character (NPC) dialogue and quest objectives, incoherent character behaviors, or violations of the game world’s internal logic. Such breakdowns largely arise from the absence of explicit state tracking or grounding mechanisms in these models. Since LLMs lack an inherent understanding of world state, quest dependencies, and previously generated content, they are prone to disrupting narrative causality and coherence. Although integrating LLMs with structured memory systems, finite state machines, or symbolic representations of world models holds promise, such hybrid approaches remain relatively underexplored within the domain of PCG, particularly for narrative tasks.

Another significant research gap exists in the evaluation of procedurally generated narratives. Unlike spatial or mechanical content—such as levels or terrains—that can be assessed using objective metrics (e.g., pathfinding solvability or object density), narrative content is inherently subjective and difficult to quantify. Current approaches often rely on human evaluations, expert annotations, or engagement proxies, which are costly, non-scalable, and lack standardization. Conventional text generation metrics like BLEU [28], and METEOR [29] offer limited utility in this domain, as they primarily measure lexical similarity rather than deeper narrative attributes such as coherence, causality, or character consistency [30].

Recent advances in natural language generation propose more nuanced evaluation methods, including structure-aware metrics, discourse modeling, and embedding-based similarity measures such as BERTScore [31]. However, these techniques remain underutilized in the context of interactive storytelling and quest generation. Key narrative dimensions—such as world-state alignment, progression logic, and character intent—are often ignored by current metrics. This highlights the pressing need for domain-specific evaluation frameworks that can robustly assess both the semantic clarity and narrative integrity of procedurally generated quests.

The integration of quest generation systems into real-time, interactive game environments presents a range of practical and technical challenges. Most LLM-based quest generation experiments are conducted offline or in isolated testing environments, without the constraints of live player interaction. For games with dynamic states, generated quests must work in tandem with the ever-evolving game state that encompasses various factors such as available inventory, completed objectives, and player positioning. This signifies the need for a bidirectional interface between the generative model and the underlying game engine. The development of such tightly coupled systems poses ongoing challenges

Scope Aspect	Description
Dataset Construction	Focused on structuring quest and dialogue data extracted from a select set of classic RPGs using prompt-aligned, XML-like annotations.
Modeling Approach	Applies quantized causal LLMs with parameter-efficient fine-tuning (LoRA) to enable low-resource adaptation to the domain.
Task Objective	Targets single-instance procedural quest generation with coherent tasks and minimal dependencies across sessions.
Evaluation Strategy	Combines automated metrics (BLEU, METEOR, BERTScore, perplexity) with human-assessed attributes (goal alignment, coherence, diversity).
Application Domain	Tailored toward fantasy-themed role-playing games with linear or semi-branching quest structures.

Table 1.1: Scope of the PQG system, including key features and constraints

Limitation	Description
Dataset Scope	The dataset is restricted to a small number of RPGs, i.e., 8, which may limit genre diversity and model generalizability.
Model Capacity	Fine-tuning is performed on quantized LLMs, potentially reducing expressiveness compared to full-precision, high-parameter models.
Dialog Context Management	The system does not track persistent dialogue state, leading to isolated quest generations without long-term context awareness.
Narrative Depth Evaluation	Evaluation focuses on surface-level lexical similarity and short-range coherence, omitting in-depth narrative or plot arc consistency.
Genre Generalization	The model and prompt format are optimized for fantasy RPGs and require more adaptation for sci-fi, modern, or non-narrative games.

Table 1.2: Summary of key limitations observed in the proposed PQG approach

in both computational design and real-time narrative integration.

The issue of scalability and performance is equally important. Large-scale models are computationally expensive and often unsuitable for real-time generation on consumer hardware. While techniques like model quantization and PEFT (e.g., LoRA adapters) offer promising trade-offs between performance and quality [8], their use in long-form, context-sensitive narrative generation has not been extensively benchmarked. The question regarding the fidelity and controllability of such compressed models in creative applications remains to be answered.

Finally, limited research has been conducted on personalization and adaptive quest generation. Existing systems typically produce static quest content that fails to account for a player’s prior actions, decisions, or preferences. Developing adaptive systems capable of tailoring quests to individual player profiles holds potential for enhancing engagement and narrative richness. However, building such a system would require robust user modeling and dynamic content adaptation mechanisms.

1.3 Proposed Solution

This study proposes a modular and resource-conscious system for PQG using quantized LLMs augmented with PEFT techniques. The solution is designed to function within the constraints of limited computational resources while ensuring semantic coherence and domain relevance in the generated quest descriptions.

At its core, the system employs causal LLMs (e.g., GPT- or LLaMA-based architectures) that have been quantized to 4-bit or 8-bit precision to reduce memory overhead and

enable deployment on modest hardware. These models are further adapted to the task domain using LoRA-based fine-tuning, which allows for task-specific training with minimal updates to the model’s core parameters. This dual strategy of quantization and PEFT balances the trade-off between efficiency and performance, particularly in low-resource environments.

To ensure alignment with RPG-specific narrative structures, the input data is curated from a domain-relevant corpus of classic RPGs. The dataset is preprocessed into an XML-like schema to reinforce consistent structure during generation and facilitate prompt-based conditioning. This schema allows the model to learn not only the linguistic form of quest descriptions but also their functional components, such as goals, rewards, and preconditions.

The quest generation task is framed as a single-instance process, wherein each quest is produced in isolation, i.e., without maintaining dialogue history or tracking long-term narrative dependencies. This design choice streamlines the modeling requirements while offering a stable and reproducible setting for model-to-model comparison.

The evaluation framework adopts a hybrid approach, combining automatic text generation metrics with human judgment. Quantitative assessment relies on widely used NLG metrics, including BLEU, METEOR, BERTScore, and model perplexity, which collectively evaluate lexical overlap, semantic fidelity, and fluency. These are complemented by qualitative human evaluations focused on narrative coherence, goal relevance, and stylistic diversity of the generated quests.

A summary of the methodological components is provided in Table 1.1, while Table 1.2 outlines the known constraints of the implementation. Together, these illustrate both the practicality and the boundaries of the proposed system.

Ultimately, this solution offers a replicable and scalable framework for evaluating the narrative capabilities of LLMs in a focused gaming context. By combining lightweight adaptation techniques with structured data and robust evaluation, the system contributes a baseline for future research in quest generation and narrative modeling.

1.4 Problem Statement

The design and delivery of compelling quest content in RPGs pose both narrative and technical challenges. Manually authored quests, while effective in delivering structured and immersive storytelling, are inherently limited by scalability, production overhead, and a lack of variability across playthroughs. PCG offers an avenue for addressing these limitations, however, current systems often struggle to produce quests that are simultaneously narratively coherent, mechanically relevant, and context-aware. LLMs, including GPT and LLaMA, have demonstrated impressive capabilities in general NLG. In particular, the relationship between model size, computational efficiency, and narrative quality in real-time game environments remains underexplored, presenting a major obstacle to the wider adoption of LLM-based systems in game development pipelines.

Thus, this study aims to bridge that gap by systematically evaluating the effectiveness of LLMs in generating semantically rich and narratively coherent quest descriptions. Unlike prior research that often relies on anecdotal evidence or game-specific testing, this work adopts a more rigorous evaluation framework using established NLG metrics, such as BLEU, METEOR, BERTScore, and perplexity, to assess multiple dimensions of language quality. Additionally, this research addresses the practical challenge of deploying

LLMs in hardware-constrained environments common in game development. It investigates the use of PEFT methods like LoRA and model quantization to reduce memory usage and computational demands without significantly compromising output quality. By integrating these optimization techniques into a low-resource generation pipeline, this study contributes both methodological insights and practical tools for scalable, real-time PQG using LLMs.

1.5 Objectives & Contributions

The overarching goal of this study is to investigate the viability of LLMs for PQG in RPGs, particularly under constraints of data availability and computational resources. To this end, the research defines five key objectives, each targeting a specific challenge in the domain of narrative generation using LLMs:

1. Curate a structured dataset from classic RPGs, annotated with quest tasks, character roles, and narrative dependencies. This supports prompt-based LLM training by exposing models to rich, domain-specific narrative patterns.
2. Analyze the performance, memory efficiency, and narrative generation quality of quantized LLMs to assess their suitability for real-time, low-resource game environments.
3. Implement PEFT using LoRA adapters to adapt base models to the RPG domain while minimizing training overhead and hardware requirements.
4. Design a comprehensive evaluation strategy combining automatic metrics (BLEU, METEOR, BERTScore, perplexity) with human-centric qualitative assessments to capture both surface-level and narrative-level coherence.
5. Construct and benchmark an end-to-end PQG pipeline integrating optimized and tagged datasets, quantized models, and LoRA-based fine-tuning to demonstrate high-quality quest generation in constrained environments.

Collectively, the successful fulfillment of the research objectives results in several novel contributions to PQG using LLMs. These include the creation of a prompt-aligned RPG quest dataset tailored for narrative learning, an empirical evaluation of quantized LLMs, and the application of LoRA-based PEFT to enhance model adaptability with minimal computational overhead. Additionally, a hybrid evaluation framework combining lexical and narrative-level metrics is introduced to enable more nuanced assessments of generation quality. Finally, the study presents a low-resource pipeline capable of generating semantically valid and narratively coherent quests, demonstrating the feasibility of integrating LLM-based systems into real-time, resource-constrained game environments.

1.6 Rest of the Report

The remainder of this report provides a structured walkthrough of the technical background, methodology, evaluation, and outcomes of the study. It introduces the core concepts behind large language models and their adaptation for narrative generation, followed by the system’s design and training workflow. The results are then analyzed using

a combination of quantitative and qualitative metrics. The structure of the report is as follows:

- "Chapter 2: Technical Background" provides the theoretical foundation for the study. It introduces LLMs used in this work (GPT-2, LLaMA 3.2, TinyLLaMA), and explains the principles of PEFT (using LoRA), model quantization techniques, transformer-based text generation, and the design logic behind PQG in RPGs.
- "Chapter 3: Related Work" surveys existing literature in PCG, quest generation using NLP techniques, and evaluation strategies for generated narrative content.
- "Chapter 4: Methodology" details the end-to-end workflow for procedural PQG, covering dataset construction, model setup, quantization, LoRA integration, and the evaluation framework. It describes how the system was designed to balance efficiency with generation quality in constrained settings.
- "Chapter 5: Results and Discussion" presents the empirical findings from model training and evaluation. It includes quantitative results (e.g., BLEU, METEOR, BERTScore, perplexity), qualitative insights, and a comparative analysis with previous studies.
- "Chapter 6: Conclusion and Future Work" summarizes the contributions of the thesis, outlines limitations, and suggests directions for future research in scalable, adaptive quest generation.
- Appendices A and B provide additional details about the quest dataset format and experimental setup, including hardware, software, and hyperparameter configurations.

This structure provides a clean and logical progression from foundational knowledge to applied experimentation techniques. Thus, it yields insights that inform both academic research and practical implementations in game development workflows.

Chapter 2

Technical Background

In this chapter, the foundational concepts and technologies underpinning the proposed system for procedural quest generation are presented. This includes an overview of LLMs, their architectural properties—particularly autoregressive transformer-based models—and their adaptation through quantization and PEFT methods such as Low-Rank Adaptation (LoRA). The chapter also explores PCG principles in the context of RPGs, detailing how narrative structures, dialogue systems, and quest schemas are typically modeled. Relevant evaluation metrics for natural language generation tasks are described to support the rationale for the chosen assessment methods. This background provides the necessary technical context for understanding the system design and experimental procedures discussed in subsequent chapters.

2.1 Large Language Models (LLMs)

LLMs are neural architectures trained on large corpora of text data to model the probability distribution of token sequences. These models are typically based on transformer decoder architectures introduced by Vaswani et al. [5], and are designed to generate text autoregressively—that is, one token at a time conditioned on the previous context. The models used in this study—GPT-2, LLaMA 3.2, and TinyLlama—are causal LLMs suitable for text generation tasks, including PQG, and vary in scale, training data, and intended deployment scenarios.

This text generation task is approached by learning to approximate the conditional probability distribution over token sequences, allowing the model to produce coherent and contextually appropriate text by predicting each token given all previous tokens in the sequence:

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t | x_1, x_2, \dots, x_{t-1}) \quad (2.1)$$

where x_t is a token at time step t , and the model is optimized to minimize the cross-entropy loss between predicted and actual tokens over large training corpora. The cross-entropy loss for a sequence is defined as:

$$\mathcal{L} = - \sum_{t=1}^n \log P(x_t | x_1, x_2, \dots, x_{t-1}) \quad (2.2)$$

Property	GPT-2	GPT-2 Medium	GPT-2 Large
Parameters	124M	355M	774M
Layers (Transformer Blocks)	12	24	36
Hidden Size	768	1024	1280
Attention Heads	12	16	20
Context Window	1024 tokens	1024 tokens	1024 tokens
Activation Function	GELU	GELU	GELU
Positional Encoding	Absolute	Absolute	Absolute
Attention Type	Self-Attention	Self-Attention	Self-Attention
Layer Normalization	Pre-LN	Pre-LN	Pre-LN
Training Dataset	WebText ($\sim$ 40GB)	WebText ($\sim$ 40GB)	WebText ($\sim$ 40GB)

Table 2.1: Comparison of GPT-2 model variants in terms of parameters

This loss penalizes low probabilities assigned to the correct next token and encourages the model to assign higher probabilities to ground truth sequences.

2.1.1 GPT-2

GPT-2 is a unidirectional transformer decoder-only model developed by OpenAI [32], designed specifically for autoregressive language modeling. It comes in multiple sizes—GPT-2 (124M), GPT-2 Medium (355M), and GPT-2 Large (774M)—all of which are used in this study. The architecture features Layer Normalization (LayerNorm), which stabilizes training and accelerates convergence by normalizing activations across features, and GELU (Gaussian Error Linear Unit) activations, which offer smooth, non-linear transformations better suited to deep transformer networks than ReLU.

GPT-2 uses multi-head masked self-attention, and absolute positional embeddings, and lacks both an encoder and bidirectional context, making it strictly left-to-right in token prediction. It is pre-trained on the WebText corpus ($\sim$ 40GB of curated web content), enabling strong performance in fluent language generation, stylistic consistency, and few-shot learning. Its smaller size variants and open licensing make GPT-2 especially suitable for low-resource fine-tuning scenarios. The comparison among the selected GPT-2 variants is given in Table 2.1.

2.1.2 LLaMa 3.2

The LLaMA (Large Language Model Meta AI) 3.2 is a collection of pretrained and instruction-tuned generative models in 1B and 3B sizes for multilingual dialogue use cases, including agentic retrieval and summarization tasks. It is an auto-regressive language model that uses an optimized transformer architecture, where the tuned versions use supervised fine-tuning (SFT) and reinforcement learning with human feedback (RLHF) to align with human preferences for helpfulness and safety [20, 33].

Llama-3.2-1B-Instruct is a lightweight version of Meta’s LLaMA-3 series, trained on a refined mixture of publicly available datasets. It incorporates recent advancements in pretraining methods and scaling strategies. This 1B-parameter model is used for instruction-following tasks, and its architecture balances generation quality with compute efficiency. The quantized version is trained on 8K context length and optimized for downstream instruction tuning tasks, making it appropriate for structured prompt-based

Property	GPT-2	Llama-3.2-1B-Instruct
Model Type	Decoder-only Transformer	Decoder-only Transformer
Parameters	124M-774M	1.23B
Hidden Size	768-1280	2048
Attention Heads	12-20	32
Context Window	1024 tokens	8,192 tokens
Activation Function	GELU	SwiGLU
Positional Encoding	Absolute	Rotary Positional Embeddings
Attention Type	Multi-head Masked Self-Attention	Grouped Query Attention
Layer Normalization	Pre-LayerNorm	RMSNorm
Training Data	WebText (~ 40 GB)	Mixture of internet corpora
Parallelism Support	Limited (older design)	Designed for better hardware efficiency (FlashAttention, grouped-query attention)
Instruction Tuning	Not included by default	Instruction-tuned variant

Table 2.2: Comparison of GPT-2 model variants with LLaMA-3.2-1B-Instruct in terms of size, architecture, and instruction-following ability

quest generation. The comparison between the GPT-2 and LLaMA models is listed in Table 2.2.

2.1.3 TinyLlama

TinyLlama-1.1B-Chat-v1.0 is a compact, instruction-tuned LLM designed to achieve competitive performance under limited compute and memory constraints. It adopts a LLaMA-style decoder-only transformer architecture, incorporating rotary positional embeddings (RoPE) for improved generalization over longer contexts and SwiGLU activations for better nonlinear representation learning. The model consists of 22 transformer layers, 32 attention heads, and a total of 1.1 billion parameters.

TinyLlama [34] is pre-trained on a curated corpus of approximately 3 trillion tokens, which includes a mix of web-scale and instruction datasets, enabling strong generalization and alignment capabilities. It uses grouped-query attention (GQA) to reduce memory usage and inference latency—especially relevant in chat-style, single-turn generation tasks [33, 35].

It is specifically optimized for instruction-following and dialogue generation and performs reliably in low-resource environments such as mobile devices or edge deployments. Its small footprint makes it a suitable backbone for PCG pipelines where efficiency, fast decoding, and high-fidelity structured output are essential.

2.2 Parameter-Efficient Fine-Tuning (PEFT)

As LLMs scale into billions of parameters, fine-tuning them for domain-specific tasks using traditional methods becomes increasingly impractical due to the high computational cost, storage requirements, and memory footprint. Standard fine-tuning modifies all model parameters, which results in expensive gradient computations and leads to multiple full-size model copies for task-specific variants. This becomes prohibitive, particularly in settings with limited resources or when maintaining multiple downstream models.

Method	Description	Trainable Params	Inference Overhead
Adapters	Inserts small feedforward modules between transformer layers. Only these modules are updated.	Low	Moderate
BitFit	Fine-tunes only bias terms in transformer layers. Surprisingly effective.	Very Low	Negligible
Prompt Tuning	Learns a set of virtual token embeddings prepended to inputs. Effective for fixed-task prompts.	Very Low	Negligible
Prefix Tuning	Adds learned key-value prefix pairs to the attention mechanism across layers. Better performance than prompt tuning.	Low	Moderate
LoRA (Low-Rank Adaptation)	Injects low-rank updates into attention projection weights (typically Q and V), keeping the base model frozen.	Very Low	Negligible

Table 2.3: Overview of common PEFT techniques used for model adaptation

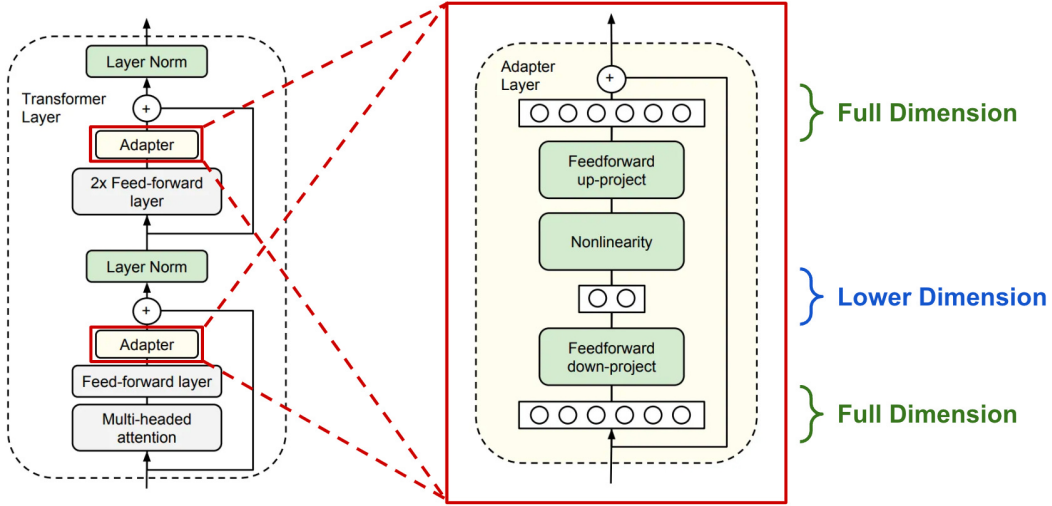


Figure 2.1: Adapter module integration by Houlsby et al. [36]

PEFT techniques mitigate this by freezing the pre-trained model weights and introducing lightweight, trainable components that allow for task adaptation without updating the entire model. These techniques are particularly suited for LLMs where full fine-tuning is prohibitively expensive in terms of memory, compute, and storage. PEFT methods reduce the number of trainable parameters, enable faster convergence, and allow storage-efficient deployment by preserving a shared backbone model and learning only small, modular adapters for downstream tasks. These approaches are critical in low-resource environments or in scenarios where multiple task-specific models are needed from a shared base. Some common PEFT techniques are described in Table 2.3.

2.2.1 LoRA: Low-Rank Adaptation of LLMs

Low-Rank Adaptation (LoRA) is a PEFT method that introduces low-rank trainable matrices into the weight structure of a pre-trained model, allowing task-specific adaptation without updating the full parameter space [8]. Traditional full fine-tuning methods

modify all model weights, which becomes infeasible for large-scale models due to high memory and storage costs. LoRA addresses this by freezing the original model weights and learning low-rank decomposition matrices that are injected into specific parts of the architecture—commonly the query and value projections in the attention mechanism.

LoRA works by expressing the weight update ΔW as a product of two smaller matrices, $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where $r \ll \min(d, k)$. The modified forward pass becomes:

$$Wx \rightarrow Wx + \alpha(AB)x \quad (2.3)$$

Here, α is a scaling factor (typically a small value like 16 or 32) and AB represents the trainable rank- r update. This decomposition drastically reduces the number of parameters introduced during fine-tuning. For instance, instead of learning a 768×768 matrix (over half a million parameters), LoRA may only learn a 768×8 and 8×768 pair (just over 12K parameters), which is more than 40x fewer parameters.

A key benefit of LoRA is compatibility with pre-trained model checkpoints. Since LoRA does not alter the base weights, the original model can be shared across tasks, while task-specific LoRA weights can be stored and applied modularly. This facilitates multi-task learning, efficient deployment, and on-device adaptation, especially useful in resource-constrained environments.

LoRA has been widely adopted in modern transformer-based models due to its effectiveness in low-resource training, and its flexibility makes it suitable for chat models, instruction tuning, and domain adaptation tasks. Moreover, recent extensions such as QLoRA further adapt this principle to quantized models, making LoRA viable even in 4-bit or 8-bit precision regimes [9].

2.3 Model Quantization

Model quantization is a technique used to compress large neural networks by reducing the precision of the weights and/or activations, which helps in reducing the memory requirements and improving computational efficiency during inference. Quantization typically involves converting floating-point numbers, which are 32-bits or 16-bits in standard neural network implementations, into lower-bit representations, such as 8-bit or even 4-bit integers. This transformation is important for deploying LLMs in environments with limited computational resources, such as mobile devices, embedded systems, or GPUs with limited memory.

The process of quantization generally involves two steps: model weight quantization (the conversion of model parameters to a lower precision) and activation quantization (the quantization of intermediate activations during inference). Quantizing both weights and activations allow the model to run with much lower memory and computational overhead, but it introduces challenges in maintaining the model’s performance and accuracy.

Quantization is especially relevant in the context of PCG and other generative tasks, where large models are typically required to generate diverse and coherent outputs. By applying quantization, it is possible to optimize these models for real-time generation without excessive resource demands.

Technique	Memory Reduction	Accuracy Impact	Use Case
8-bit Quantization	~4x reduction from FP32	Low-to-medium	General purpose, widely adopted for LLMs
4-bit Quantization	~8x reduction from FP32	High loss	Extremely low-resource environments

Table 2.4: Common quantization methods used to compress and accelerate LLMs

Benefit	Description
Memory Efficiency	Reduced model size, enabling deployment in resource-limited environments.
Faster Inference	Lower precision enables faster computation, especially on optimized hardware.
Low-Resource Deployment	Makes large models feasible on mobile, embedded, and edge devices.

Table 2.5: Key benefits of applying quantization to language models

2.3.1 Quantization Techniques (e.g., 4-bit, 8-bit)

Quantization techniques can be categorized based on the level of precision and how the quantization is applied to the model. The most common techniques are 8-bit quantization and 4-bit quantization, each with its benefits and trade-offs. These techniques are crucial for reducing the memory footprint of models and enabling efficient deployment in resource-constrained environments.

8-bit quantization (INT8) is one of the most widely used techniques in model compression. It reduces the precision of the model’s weights and activations to 8 bits, which is a significant reduction from the typical 32-bit floating-point precision. This technique is generally effective at achieving a good balance between memory efficiency and model performance.

4-bit quantization (INT4) reduces the model weights and activations to just 4 bits. This technique further reduces the memory footprint and computational requirements, making it particularly useful for extremely resource-constrained devices. However, 4-bit quantization can introduce significant accuracy loss, and it is often more difficult to implement without specialized hardware.

The main goal of these quantization techniques is to achieve the best trade-off between efficiency and accuracy. A summary of the different approaches to model quantization are given in Table 2.4.

2.3.2 Benefits and Trade-offs

The key benefits of quantization include significant memory savings and faster inference times, especially when deploying large models for inference in resource-constrained environments. However, there are trade-offs, particularly with respect to model accuracy. Quantization tends to introduce some degree of error due to the reduction in precision, which can affect the generation quality in tasks requiring high accuracy and fluency. Various benefits and trade-offs of employing model quantization techniques are listed in Table 2.5 and Table 2.6 respectively.

Trade-off	Description
Accuracy Loss	Reduced precision can negatively impact generation quality, especially for tasks requiring fine-grained language understanding.
Training Complexity (QAT)	Quantization-aware training can be resource-intensive and time-consuming.
Hardware Dependency	Benefits are dependent on hardware capabilities, which may not support low-precision operations.

Table 2.6: Analysis of trade-offs introduced by model quantization

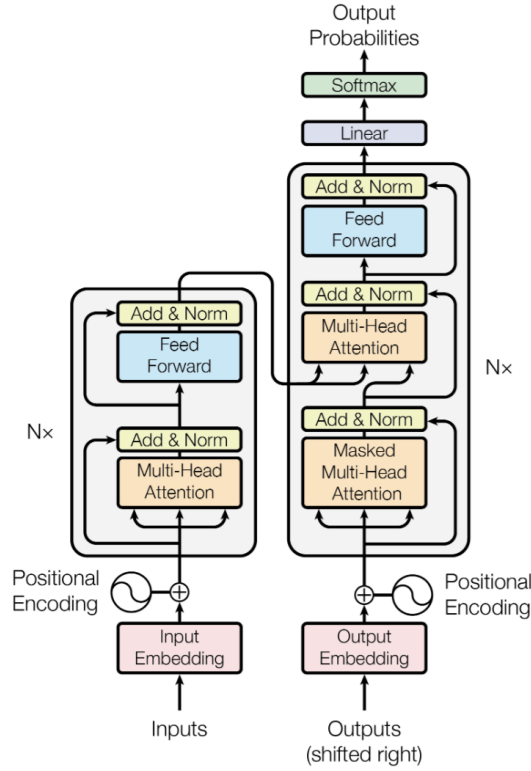


Figure 2.2: Transformer architecture by Vaswani et al. [5]

2.4 Transformers and Text Generation

Transformer architectures, introduced by Vaswani et al. (2017) [5], revolutionized NLP by replacing recurrence with attention mechanisms, enabling parallel computation and better handling of long-range dependencies. The core of the Transformer is the self-attention mechanism, which allows each token to weigh the importance of other tokens in a sequence dynamically. This structure significantly improves scalability and representational capacity, forming the backbone of most modern LLMs.

Transformers consist of stacked encoder and/or decoder layers. For autoregressive text generation tasks, models typically adopt a decoder-only architecture, as seen in GPT-style models. These models predict the next token in a sequence based on the previously generated tokens, learning the conditional probability distribution over sequences as seen in Equation 2.1.

Each decoder block includes multi-head masked self-attention (to preserve causality), feed-forward layers, residual connections, and normalization layers (typically LayerNorm). In the context of text generation, the decoder-only transformer model is trained to mini-

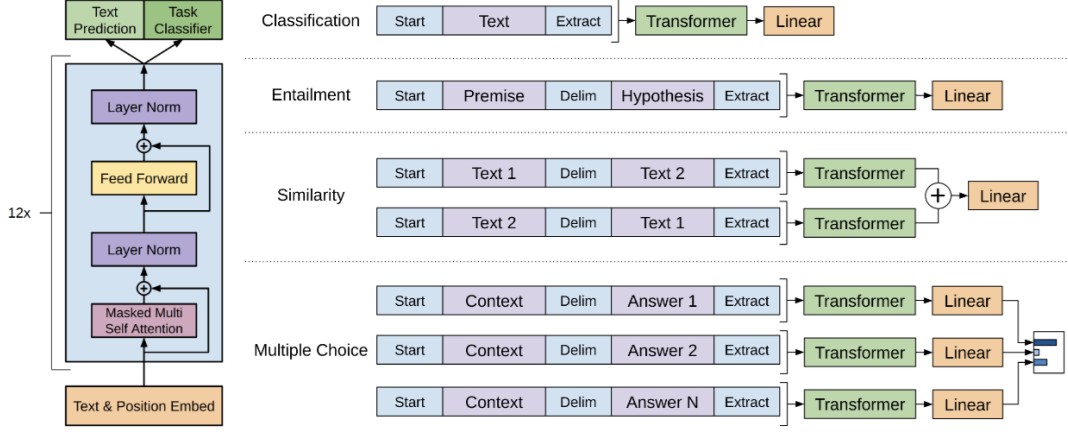


Figure 2.3: Transformer and autoregressive generation using tokenized inputs and a softmax output layer by Radford et al. [37]

minimize the cross-entropy loss between predicted and actual tokens across large-scale corpora as seen in Equation 2.2.

This optimization enables the model to learn language structure, semantics, and context, empowering it to generate coherent, contextually grounded text. During inference, greedy decoding, beam search, or sampling-based strategies (e.g., top-k, nucleus sampling) are employed to generate sequences token by token. The quality of generated text is influenced by model size, training data diversity, and decoding strategy.

2.4.1 Causal Language Modeling

Causal language modeling (CLM) is the fundamental training paradigm for decoder-only transformer architectures, where the model is trained to predict the next token in a sequence based solely on the tokens that precede it. It is termed causal because the self-attention mechanism is masked such that each token can only attend to earlier positions in the input sequence, strictly preventing any future context from influencing current predictions. This autoregressive nature ensures that the generation process proceeds one token at a time, maintaining a left-to-right structure.

The CLM objective is defined as the maximization of the joint likelihood over a sequence by decomposing it into a product of conditional probabilities as defined in Equation 2.1.

This modeling approach encourages the network to develop a deep understanding of linguistic dependencies and long-range structure in text. During training, the model is optimized using cross-entropy loss, comparing the predicted probability distribution with the actual next token.

CLM is particularly suited for generative tasks where output sequences must be fluent, coherent, and sensitive to prior context. It has found significant application in storytelling, dialogue generation, and PCG for games, where maintaining narrative consistency and character-specific voice across generated outputs is critical. Moreover, because the model generates text one token at a time, it is naturally aligned with inference-time decoding techniques such as greedy decoding, beam search, and sampling, allowing fine-grained control over generation behavior.

2.4.2 Tokenization and Prompt Formats

Before training or inference, raw text must be tokenized into discrete units understood by the model. Most transformer-based LLMs use subword tokenization strategies like byte pair encoding (BPE) [38], WordPiece [39], or SentencePiece [40]. These techniques balance vocabulary size with the ability to represent rare or unknown words through compositional subunits. GPT-2 uses BPE-based tokenization with a vocabulary of $\sim 50,000$ tokens, whereas LLaMA models typically use SentencePiece with byte-level fallback for multilingual robustness.

Prompt formatting plays a crucial role in guiding autoregressive generation. A well-structured prompt helps align the model’s generation behavior with task-specific needs. For PCG, especially for quests and dialogue, the structured XML-like format we use in this project provides an explicit and parseable schema that delineates each component of the generated content, such as `<|begin_quest|>`, `<|objective|>`, `<|quest_giver|>`, and `<|end_quest|>`. This format not only improves model interpretability but also ensures consistency across different samples by providing strong inductive biases during both training and inference.

By enforcing a rigid prompt structure, we reduce the variability in input-output mappings and facilitate easier post-processing and validation. Furthermore, the format aids in fine-tuning and evaluation, allowing automatic extraction of key fields without relying on fragile heuristics. The inclusion of special tokens and structural boundaries also conditions the model more effectively, promoting accurate task alignment, especially in constrained generation scenarios like low-resource training or quantized models.

This approach is particularly beneficial for domain-specific applications such as RPG quest generation, where narrative components must adhere to recognizable patterns. Structured prompts help the model disambiguate semantic roles and maintain coherence, even in zero-shot or few-shot settings.

2.5 Procedural Quest Generation

PQG is the computational process of automatically generating quests (structured narrative tasks or missions) for players within a digital game environment. Unlike manually authored quests, which are created by game designers, PQG enables the dynamic and scalable creation of narrative content through algorithmic methods. The core motivation behind PQG is to reduce authoring costs, increase content variability, and improve replayability, especially in open-world or sandbox-style games.

2.5.1 Structure and Components of a Quest

In game design, a quest refers to a carefully constructed sequence of tasks or challenges assigned to the player, designed to fulfill a specific in-game objective. Quests are foundational to many games, especially RPGs, as they provide narrative context, gameplay direction, and a sense of purpose. They are often embedded with storytelling elements, gameplay mechanics, and incentive structures, combining to create engaging and meaningful player experiences.

Quests typically begin with actors, often non-player characters (NPCs), who serve as quest givers or participants in the unfolding events. These characters help establish the quest in the game world such that they deliver dialogue, offer context, or accompany the

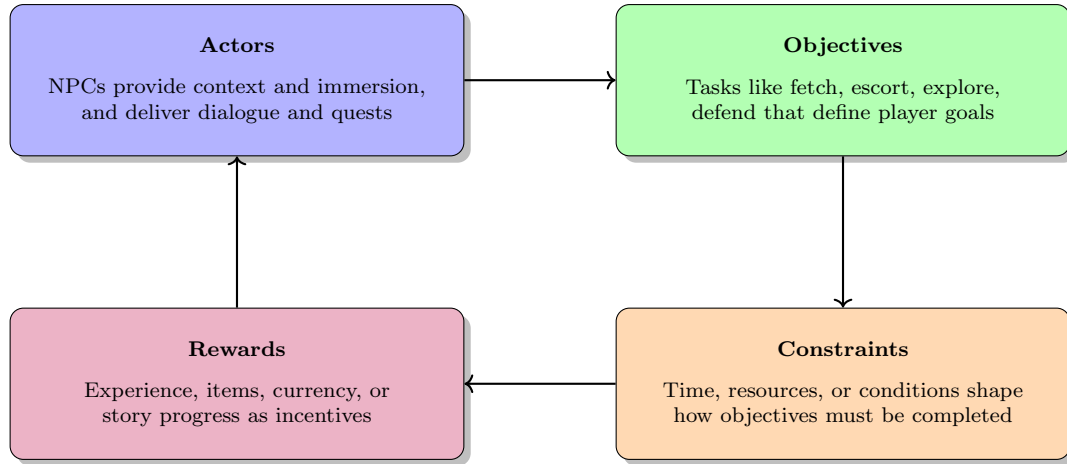


Figure 2.4: Circular flow diagram illustrating the core components of a quest in game design: actors, objectives, constraints, and rewards. Each component plays a critical role in shaping player experience and quest structure.

player on their journey. The interactions of these quest givers with the player reinforce the immersion within the game’s storyline and explain the worldbuilding.

At the core of each quest are the objectives that are the specific tasks the player is expected to accomplish. These can range from common actions like retrieving a lost item (“fetch quests”) and guiding a character safely through danger (“escort quests”) to more dynamic challenges such as defending a location, exploring uncharted areas, or eliminating threats. Quests often combine multiple objective types in a sequence to form multi-step missions. This enhances the gameplay complexity and variety.

Quests are further tied to the constraints that influence how objectives must be achieved. These may include time restrictions, resource limitations, or conditional requirements for success. For example, a quest may need to be completed before nightfall, using only a limited set of tools, or without being detected. Such constraints introduce strategic thinking, increase the difficulty of the gameplay, and differentiate one quest experience from another.

Upon completion, quests provide rewards as incentives for the player’s effort. These rewards may take various forms, including experience points (EXP) that contribute to character progression, valuable in-game items, currency, or narrative developments such as unlocking new characters or advancing the storyline. The reward structure often correlates with the complexity or risk associated with the quest, reinforcing the player’s sense of accomplishment.

Altogether, these four elements, namely actors, objectives, constraints, and rewards, constitute the fundamental building blocks of a quest [41]. A deep understanding of these components is not only vital for traditional quest design but also forms the basis for automating the generation of quests through procedural methods.

2.5.2 Procedural Generation in Quest Design

Procedural generation is a powerful content creation paradigm widely used in video games to algorithmically produce environments, characters, and other game assets, often enabling large-scale variability and replayability. PQG extends this concept to the domain of quest design, aiming to automate the creation of quests that are not only structurally

Approach	Method Used	Narrative Coherence	Flexibility	Example Systems
Rule-based	Predefined rules and templates combining structured quest components	Moderate - structured but often lacks narrative depth	Low - limited variation leads to repetition	Classic dungeon crawlers, Skyrim radiant quests [42]
Planning-based	AI planning methods like STRIPS to generate goal-driven quest sequences	High - ensures logical progression and goal alignment	Moderate - adapts to context but constrained by planner limits	STRIPS-based planners [43], PaSSAGE [44]
Grammar-based	Formal grammars and domain-specific ontologies to define quest structure	High - formalizes quest logic and relationships	Moderate - dependent on predefined models and coverage	Mission graph system [45], Doran & Parberry’s quest generator [46]
Generative models	Data-driven models and LLMs for narrative generation	Very High - produces rich, context-aware stories	High - adaptable to a wide range of themes and player inputs	GPT-based quest generators, AI Dungeon [4]

Table 2.7: Comparative analysis of common PQG methods

diverse but also narratively engaging and contextually appropriate within the game world.

Various computational techniques are employed in PQG to achieve this goal. One common approach is the use of algorithmic rules or templates, which define predefined structures or patterns that quests can follow. These templates can be customized with variable content, allowing for a wide range of quest variations while maintaining logical consistency.

Another technique involves planning algorithms, typically drawn from AI, which simulate goal-driven behavior by generating sequences of actions that lead from an initial game state to a desired outcome. This enables the generation of quests that mimic intentionality and logical progression, often making them feel more believable and purpose-driven.

Grammars and ontologies are also used to formally define the relationships between quest components such as actions, characters, locations, and objectives. By encoding domain knowledge into structured formalisms, grammars and ontologies help ensure that generated quests are semantically coherent and compatible with the broader game world. Dormans (2010) introduced a grammar-based approach that combines mission structure and spatial layout generation through graph rewriting techniques, allowing for the creation of quests that are both logically structured and integrated with the game environment [45].

More recently, machine learning models, particularly LLMs, have been integrated into PQG systems to enhance narrative richness and human-like expression [24]. These models can generate quest dialogues, descriptions, and plot developments that are contextually nuanced and linguistically natural, elevating the player’s narrative experience.

The overarching objective of PQG is not merely to automate quest production, but to do so in a manner that maintains narrative coherence, adapts to player behavior, and aligns with the dynamic context of the game. This makes PQG a challenging yet promising area of research in PCG, where both design constraints and player engagement must be carefully balanced.

Chapter 3

Related Work

In this chapter, we examine foundational and recent advances in PCG within digital games, with a specific focus on quest generation. This includes traditional rule-based and planning approaches, search-based PCG, and the integration of deep learning and LLMs. We draw from key contributions by Togelius et al. [47] and Shaker et al. [48], who formalized PCG taxonomies and techniques, as well as Liu et al. [49], who surveyed the deep learning landscape for PCG. We also explore works by Värtinen et al. [24] and Ammanabrolu et al. [22], which apply LLMs to generate quest narratives.

Historical systems such as Player-Specific Stories via Automatically Generated Events (PaSSAGE) [44] and Thespian [50] focused on emergent, personalized storytelling through player modeling and modifiable agents. Meanwhile, approaches like REwriting Graphs for Enhanced Narratives (ReGEN) [51] and Creation of Novel Adventure Narratives (CONAN) [52] applied graph rewriting and grammars to produce narrative coherence in quest design. However, many of these techniques suffer from issues of repetitiveness and low generalizability.

Modern research increasingly leverages transformer-based LLMs, offering improved diversity and coherence. For instance, van Stegeren and Myśliwiec [25] demonstrated GPT-2’s capabilities on a World of Warcraft [53] dataset, while Värtinen et al. [24] proposed a broader dataset spanning six RPGs. Their findings suggest that LLM-generated quests can sometimes surpass human-written content in perceived quality, though sample sizes and model inputs often limit generalizability.

3.1 Procedural Content Generation in Games

PCG refers to the algorithmic creation of game content with limited or indirect user input. It has seen widespread adoption in game development for generating levels, items, characters, quests, and more. The primary motivation behind PCG is to reduce the manual effort of content creation while enabling rich, dynamic, and replayable experiences.

Search-based PCG was introduced prominently by Togelius et al. [47]. This approach treats content generation as an optimization problem, using algorithms like evolutionary strategies, genetic algorithms, and Monte Carlo tree search (MCTS) to search a space of potential content. The fitness function in such cases evaluates how “playable” or “fun” a piece of content is. Rule-based and agent-based methods offer deterministic content generation but often lack variety. For instance, cellular automata and wave function collapse are used for map generation and tiling [48].

Deep learning-based PCG techniques have expanded the scope of generative models

Author(s)	Datasets Used	Techniques Applied	Evaluation Metrics	Generation Strategy
Ammanabrolu et al. (2019) [22]	Recipe data adapted as proxy quest structure for a cooking-themed text adventure, used with the TextWorld framework [57].	A hybrid model that combined KGs (for structure), Markov chains (for progression), and LSTM RNNs (for sequences).	Human ratings for coherence, novelty, value, and unpredictability. Likert scale [58] + ANOVA + Tukey [59] post-hoc tests.	Focused on generating coherent sequences of actions in text-adventure format. Used KGs for semantic grounding and logical consistency.
Ashby et al. (2023) [23]	Dynamic knowledge graph based on RPG world state. Player-specific input encoded as graph traversals to retrieve relevant quest content.	Graph traversal to extract quest ingredients, grammar-based template filling, LLM-based title/dialogue generation.	Fluency, coherence, novelty, and creativity. Introduced lemma co-occurrence scores for semantic alignment.	Player input is processed via cosine similarity on graph relations to retrieve relevant quest paths. Classified into combat/gather/explore types and rendered via LLM-enhanced templates.
Värtinen et al. (2024) [24]	A curated dataset of 978 quest descriptions from six RPGs (e.g., <i>Baldur's Gate</i> , <i>Skyrim</i> , <i>Oblivion</i>), extracted from game files and cleaned for narrative quality.	Fine-tuning GPT-2 XL (1.5B) on narrative-format quests with placeholder substitution. Evaluated different metadata formats (XML-like, simple, narrative).	Perplexity, cross-entropy loss, human Likert ratings for coherence and fit, and heuristic filtering.	GPT-2 is fine-tuned on structured quest data by encoding as XML-like prompts. Generation is performed using these prompts to produce quest descriptions.

Table 3.1: Summary of works focused on structured quest generation techniques

by enabling learning from existing content distributions. CNNs (convolutional neural networks) and LSTMs (long short-term memory) have been used for visual and sequential content respectively [49]. Guzdial et al. [54] used CNNs for visual level blending, while Liang et al. [55] used LSTMs for beatmap generation. GANs (generative adversarial networks) and autoencoders have enabled unsupervised learning and blending of complex structures. Sarkar et al. [56] demonstrated controllable level blending using autoencoders.

3.2 NLP Techniques for Quest Generation

NLP has become integral to PCG in games, especially for generating narrative structures such as quests. Among various approaches, structured quest generation using LLMs and hybrid systems has emerged as a dominant paradigm. A notable early contribution by Ammanabrolu et al. [22] proposed a knowledge graph-based architecture for generating quests in text-adventure games. Their framework effectively combined symbolic planning, rule-based logic, Markov chains, and neural models to create coherent, goal-oriented storylines. This hybrid approach demonstrated the potential of integrating structured world representations with data-driven generation techniques.

Later, Värtinen et al. [24] introduced a comprehensive quest dataset collected from six major commercial RPGs. They used this dataset to train a GPT-2 model—referred to as Quest-GPT-2—for generating quests that reflected patterns and structures found in professionally designed game content. This dataset has since served as a benchmark for evaluating model performance in structured quest generation tasks.

Building on these foundations, recent work has shifted towards using fine-tuned or instruction-tuned LLMs that accept partially structured inputs—such as templates, quest metadata, or dialogue context—to generate diverse and coherent quest content. These models are designed to align better with human-authored quest structures, enabling the generation of narrative elements that maintain logical progression and thematic consistency.

Table 3.1 presents a comparison of representative systems for structured quest generation. It outlines their dataset sources, modeling techniques, and evaluation strategies, highlighting both the evolution of methodologies and the diversity of benchmarks used across studies.

As evident from the table, earlier models relied on handcrafted environments or symbolic planners, while recent approaches increasingly leverage large-scale pre-trained language models. However, consistent evaluation remains a challenge, with most works relying on task-specific metrics like BLEU [28] and ROUGE [60], which may not fully capture narrative quality or coherence.

3.3 Human Evaluation of Generated Content

In addition to automated metrics, human evaluation plays a crucial role in assessing the quality and effectiveness of generated quests. This method focuses on subjective aspects that are often difficult to quantify, such as coherence, creativity, and how well the generated content aligns with the game world’s narrative and logic. Human evaluations are commonly employed to capture the user experience and assess whether the generated quests feel meaningful and engaging within the context of the game.

For instance, Ammanabrolu et al. [22] used human evaluators to rate the coherence and creativity of quests generated by their hybrid knowledge graph (KG) and neural model approach. They focused on understanding how well the generated quests fit into the context of text-based adventure games and whether they maintained narrative flow. Similarly, van Stegeren et al. [25] employed participant ratings to evaluate quest quality in their study on automated quest generation for RPGs. These evaluations often include ratings on creativity, narrative alignment, and the overall immersion that the generated quests provide.

Värtinen et al. [24] also conducted a survey-based evaluation as part of their Quest-GPT-2 study. Participants were asked to compare generated quests with human-authored counterparts based on multiple dimensions, including structure, creativity, and plausibility within a game world. The results showed a preference for human-authored quests but highlighted the potential of fine-tuned language models to approximate professional design standards when guided by structured data.

Recent research in human-computer interaction (HCI) also emphasizes the importance of player-centered design when evaluating procedurally generated narratives. For example, Smith et al. [61] argue that evaluation frameworks for PCG must consider interactivity, engagement, and player agency as factors that are often overlooked in standard NLP evaluations. HCI perspectives advocate for iterative testing with diverse user groups and the integration of usability metrics to ensure that generated quests support player satisfaction and narrative immersion across different play styles and preferences.

Despite the importance of human evaluations, there are certain limitations to this approach. One common issue is the small sample size of participants, which can lead to variability in results and limit the robustness of the conclusions. Additionally, the specific

context or domain of the quests evaluated may not always generalize well to other types of games or genres. For example, a system trained and evaluated on quests for a medieval fantasy RPG may not perform as well when applied to a sci-fi or historical setting. Furthermore, the subjective nature of human evaluations can introduce biases based on individual preferences, leading to inconsistency in ratings across different evaluators.

While human evaluations remain a vital tool in quest generation research, these limitations underscore the need for a combination of quantitative metrics, HCI-driven evaluation frameworks, and subjective assessments to better understand and improve the quality of generated content.

Chapter 4

Methodology

In this chapter, we outline the methodology used to develop and evaluate the procedural quest generation system for RPGs using LLMs. The goal of this research is to create an automated framework that can generate coherent, engaging, and contextually appropriate quests for RPGs, leveraging the power of LLMs and hybrid architectures. This chapter describes the step-by-step procedure, from data collection and pre-processing to model training, fine-tuning, and evaluation. The methodology is designed to ensure that the generated quests are of high quality, both from a computational and a user experience perspective.

We begin by discussing the dataset collection and preparation, including the specifics of the quest data used to train the models. Next, we detail the architecture of the LLM-based models, the fine-tuning procedures, and the use of LoRA adapters to optimize model performance for quest generation. Following that, we delve into the evaluation metrics employed to assess the generated quests, explaining both quantitative and qualitative methods such as automatic evaluation metrics and human evaluation. Finally, we outline the experimental setup and the procedures followed during model training, validation, and testing, with a focus on how the methodology ensures the generation of diverse and contextually rich quests for RPGs.

4.1 Workflow for Procedural Quest Generation

This section provides an end-to-end overview of the PQG workflow implemented in this study. The workflow integrates multiple components, from dataset preparation to model fine-tuning and evaluation, to produce coherent, context-aware, and narratively rich quests for RPGs. Each phase of the workflow has been designed to ensure the robustness, efficiency, and adaptability of the quest generation system. The workflow is structured to align with current best practices in NLG using instruction-tuned LLMs, while addressing the unique challenges of interactive narrative construction in games.

The system emphasizes modularity across all stages, allowing rapid experimentation with models, preprocessing, and evaluation strategies. From curated, annotated quest datasets to prompt-based generation with lightweight fine-tuning, each component supports a cohesive and flexible pipeline. Quantization and parameter-efficient tuning (e.g., LoRA) enable deployment in constrained environments without compromising quality. Generation and evaluation setups are optimized to capture both lexical accuracy and semantic depth. The complete workflow is represented by the flow diagram depicted in Figure 4.1.

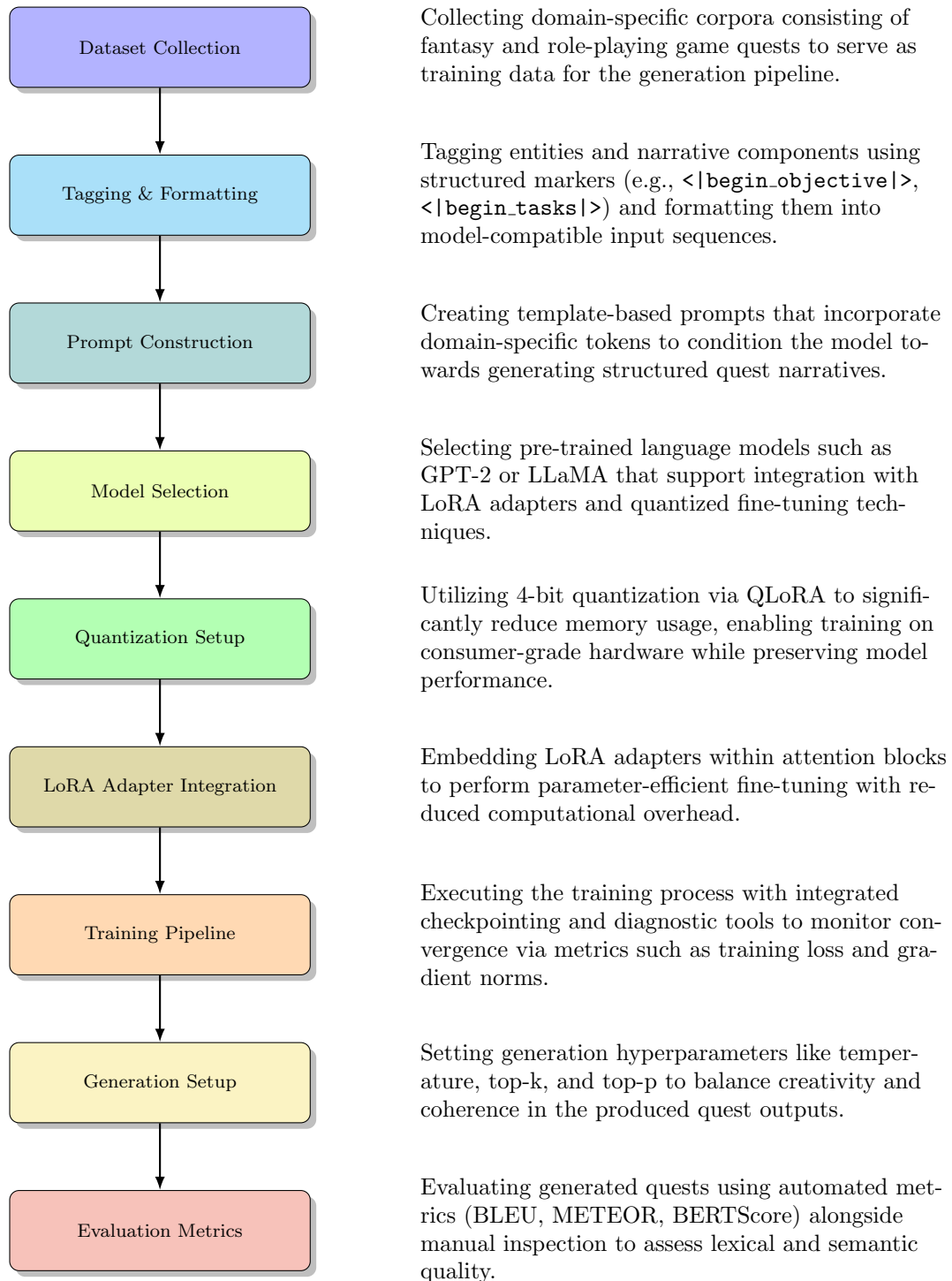


Figure 4.1: Flow diagram of the procedural quest generation system. It illustrates the modular pipeline used for model training and evaluation.

4.1.1 Dataset Collection and Formatting

The process begins with dataset preparation, a foundational step critical for training effective quest generation models. Due to the scarcity of structured, publicly available quest datasets, data is collected from a variety of sources, including open-source RPG scripts (e.g., *Skyrim* [15] and *Baldur’s Gate* [12]), community-maintained wikis, and synthetic examples generated using quest templates.

The primary dataset used in this study is the Värtinen et al. [24] dataset, which consists of 978 manually annotated quests extracted from a diverse set of RPG titles, including *Minecraft* [3] and *Torchlight II* [62]. Each quest in this dataset is represented in a structured XML-inspired format, comprising tags for key quest elements including `<|begin_objective|>`, `<|begin_tasks|>`, and `<|begin_quest_giver|>`. This format maintains the meaningful connections between quest elements while enabling efficient structured data processing for subsequent tasks.

To enhance the dataset’s scope and represent a broader variety of game narratives, this study appends 156 additional quests from the *Fallout series* [10, 11]. These quests are extracted manually from community-maintained sources and restructured to align with the XML-like format used by Värtinen et al. [24], ensuring consistency and interoperability.

After consolidation, the dataset underwent preprocessing to resolve inconsistencies and segment each quest into well-defined narrative components. The structured quests are then transformed into an instruction-style prompt-response format to suit instruction-tuned language models. A sample of the formatted quests and the prompt-response formatting can be seen in Appendix A.

4.1.2 Model Selection and Evaluation

Following dataset construction, the workflow progressed to model selection, quantization, and fine-tuning. Multiple pretrained language models are evaluated, including GPT-2 variants, TinyLLaMA, and LLaMA 3.2, to compare their performance in PQG. These models are chosen for their varying parameter scales, open availability, and relevance to instruction-based generation tasks.

To enable training on resource-constrained, consumer-grade hardware, all models are quantized to 4-bit precision using QLoRA (Quantized Low-Rank Adaptation) [9]. This method significantly reduces memory usage and computational overhead while maintaining most of the model’s generative capabilities. LoRA is used as the primary PEFT method, enabling efficient updates through low-rank projections without modifying the entire model architecture [7, 8].

Fine-tuning is conducted using the XML-inspired, instruction-annotated dataset, converted into prompt-response pairs. Although true mixed-precision training is not supported due to backend limitations, float16 precision is employed where possible to reduce memory footprint. Gradient checkpointing is utilized to further optimize memory usage during backpropagation, and early stopping is incorporated to prevent overfitting based on validation loss trends. The training objective centered on minimizing cross-entropy loss between model outputs and target quest responses, enhancing the model’s ability to produce coherent, context-aware, and narratively rich quests.

Once fine-tuning is complete, model evaluation is conducted using a combination of automated metrics and comparative performance analysis. Automated evaluation involved standard NLG metrics, including BLEU, METEOR, and BERTScore, to assess lexical overlap, semantic alignment, and fluency between generated and reference quest

narratives. In addition to these, perplexity is used as an intrinsic measure of the model’s generative confidence, with lower perplexity indicating better language modeling performance.

4.1.3 Framework Scope and Limitations

While deployment and runtime integration are not the primary objectives of this study, the system has been developed as a procedural framework for advancing quest generation in low-resource environments. The emphasis is placed on designing an adaptable pipeline that demonstrates the feasibility of generating context-aware and narratively coherent quests using quantized language models and PEFT methods. Rather than targeting production-ready deployment, the goal is to provide a robust experimental foundation that can be extended with larger models, richer datasets, and more advanced training configurations.

4.2 Dataset Description and Preprocessing

This section introduces the dataset compiled for training and evaluating LLMs in the task of PCG. The construction of the dataset involves aggregating quest data from diverse RPG sources, standardizing the structure into a uniform schema, and formatting it for compatibility with generative language models.

Given the narrative and structural complexity of quests in RPGs, it is essential to represent each quest in a machine-readable yet semantically rich format that preserves critical gameplay elements such as objectives, subtasks, quest givers, rewards, and supporting entities. This requires a preprocessing pipeline capable of handling heterogeneous data from multiple games, some of which originate from structured datasets while others are manually curated and normalized with the aid of LLMs.

The final dataset, referred to as the QUEST Dataset, captures a wide range of quest archetypes and contextual variations across both high-fantasy and post-apocalyptic game settings. It combines data from multiple titles, including games from the *Elder Scrolls*, *Fallout*, and *Baldur’s Gate* franchises [10–15], and ensures each entry adheres to a unified tagging convention that facilitates token-level alignment during training.

To prepare the dataset for sequence modeling tasks, each quest instance is transformed into a text sequence using an XML-inspired prompt structure. This formatting approach allows for consistent parsing, enhances controllability during generation, and supports both conditional and unconditional sampling in downstream experiments. The details of data collection, game-wise contributions, and the exact schema used for formatting are described in the following subsections.

4.2.1 Source and Specification of Dataset

The QUEST dataset is constructed by aggregating quest data from a curated collection of eight popular RPGs. These include titles from both Western and sandbox RPG traditions, ensuring diversity in quest structure, theme, and complexity. The primary source of this dataset is the open-access quest corpus published by Värtinen et al. [24], which includes quests from six mainstream RPGs. To enhance the dataset’s size and narrative richness, additional quests from *Fallout* [10] and *Fallout 2* [11] were manually collected and structured using a semi-automated method involving an LLM assistant.

Game Title	Source	Number of Quests	Percentage
Baldur’s Gate [12]	Värtinen et al. [24]	100	8.8183%
Baldur’s Gate II: Shadows of Amn [13]	Värtinen et al. [24]	94	8.2892%
The Elder Scrolls IV: Oblivion [14]	Värtinen et al. [24]	215	18.9594%
The Elder Scrolls V: Skyrim [15]	Värtinen et al. [24]	389	34.3034%
Minecraft [3]	Värtinen et al. [24]	100	8.8183%
Torchlight II [62]	Värtinen et al. [24]	80	7.0547%
Fallout [10]	Nukapedia [63]	58	5.1146%
Fallout 2 [11]	Nukapedia [64]	98	8.6420%
Total		1134	99.9999%

Table 4.1: Distribution of collected quest data categorized by the game title



Figure 4.2: Cover arts for selected RPGs used in the QUEST dataset [10, 13, 15]

Each quest entry is composed of a set of core and optional fields that reflect the semantic elements typically found in narrative design: goals, tasks, characters, environments, and consequences. The data schema was extended slightly from the original dataset to accommodate additional information present in the manually added entries (e.g., character motivations, groups, and auxiliary items). These data fields are clearly defined in Appendix A.

The inclusion of quests from *Fallout* [10] and *Fallout 2* [11] expanded the original dataset by 156 unique entries, enhancing both the size and variety of the corpus. Among all the titles represented, *The Elder Scrolls V: Skyrim* [15] contributes the largest share of quests, accounting for approximately 34.30% of the total, whereas *Fallout* [10] contributes the fewest, at 5.11%.

To support richer expression during generation, the dataset retains several sparsely populated or “rare” columns—fields that are not consistently filled across all game sources but are preserved for flexibility in representing optional or game-specific quest attributes. As a result, the compiled dataset offers a comprehensive and heterogeneous benchmark for evaluating the narrative coherence, structural consistency, and alignment with game-world logic in procedurally generated quests.

4.2.2 Tagging and Formatting

To enable the dataset’s compatibility with autoregressive LLMs, each quest entry is serialized into a structured, tag-based format resembling XML. This formatting step ensures consistency, simplifies prompt construction, and aligns with the model’s tokenization strategies during both training and inference.

The formatting approach employs custom tags that explicitly delineate each quest field. Mandatory components—such as `<|begin_objective|>`, `<|begin_tasks|>`, and `<|begin_quest_giver|>`—are always included to guarantee minimum structure, while optional fields are conditionally inserted based on content availability. This design accommodates the dataset’s heterogeneous nature and supports the generation of partial or context-dependent quests.

```
<|begin_quest|>
<|begin_objective|>
[Objective goes here]
<|end_objective|>
<|begin_tasks|>
[Task 1]
[Task 2]
<|end_tasks|>
<|begin_quest_giver|>
[Name]: [Description]
<|end_quest_giver|>
...
<|end_quest|>
```

Listing 4.1: Snippet of the XML-like quest formatting template

To support this transformation, a custom Python function (`format_quest_as_xml`) was developed. It parses each quest dictionary, extracts structured elements (like `rewards`, `locations`, or `enemies`), and generates a well-formatted string using a consistent template. Nested and list-based fields are joined with line breaks for model readability and separation of elements. The complete tagging and formatting script is available in the project repository¹.

This structured formatting approach plays a critical role in preparing the dataset for model training. It enforces uniform tag usage, ensuring that all quest entries—regardless of their original source—follow a standardized input format. The design also offers flexibility, allowing non-essential fields to be selectively included or omitted without affecting structural integrity.

Additionally, it facilitates controlled prompt construction, enabling fine-tuned exposure of specific quest attributes during training and evaluation. By converting diverse, game-specific data into a consistent and model-compatible format, this step serves as a vital bridge between raw input and effective language model utilization. Further specifications for the formatting procedure can be seen in Appendix A.

4.3 Model Configuration and Training

This section describes the architecture, configuration, and training procedures used to fine-tune language models for the task of PQG. Given the size and complexity of modern LLMs,

¹<https://github.com/DracoY-code/quest-generation>

Parameter	Value
Quantization Type	4-bit NormalFloat (NF4)
Compute Data Type	bfloat16 / float16
Double Quantization	Enabled
Library Used	bitsandbytes with Hugging Face Transformers

Table 4.2: Quantization configurations used in model compression for training

Parameter	Value
Target Modules	Attention projection layers (q_proj, v_proj, o_proj)
Rank (r)	4
Alpha (α)	8
Dropout	0.0
Bias	None
Library Used	PEFT (Parameter-Efficient Fine-Tuning)

Table 4.3: Configuration of LoRA adapter modules integrated during fine-tuning

this phase incorporates efficient training strategies such as quantization and LoRA [8] to make the process more computationally tractable and memory efficient.

4.3.1 Quantization Workflow

To reduce memory usage and accelerate training, model weights were quantized using the BitsAndBytes² library. This allowed loading larger models into memory-constrained environments while preserving inference quality. The quantization configuration is based on the 4-bit NormalFloat (NF4) format with double quantization and computation performed in 16-bit precision (bfloat16 or float16).

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)
```

Listing 4.2: Quantization configuration using BitsAndBytes with 4-bit NF4 precision

This setup, as seen in Table 4.2, significantly reduces the GPU VRAM footprint without degrading downstream performance, enabling multi-billion parameter models to be trained on consumer-grade hardware or limited cloud instances.

4.3.2 LoRA Adapter Integration

To further reduce the number of trainable parameters, Low-Rank Adaptation (LoRA) [8] modules were employed. LoRA injects trainable low-rank matrices into the attention layers of transformer models, allowing efficient adaptation without modifying the base model weights.

LoRA was selected for its efficiency and flexibility in fine-tuning LLMs. It enables parameter-efficient training by updating only a small subset of parameters—typically less

²<https://github.com/bitsandbytes-foundation/bitsandbytes>

Feature	Details
Frameworks	<code>transformers</code> , <code>datasets</code> , <code>accelerate</code> , <code>peft</code>
Trainer	<code>transformers.Trainer</code> API
Tokenization	Custom tokenizer with special XML-like tokens (e.g., <code>< begin-objective ></code>)
Loss function	Cross-entropy
Evaluation	Perplexity, BLEU, METEOR, BERTScore on a held-out test split
Logging	Optional integration with TensorBoard

Table 4.4: Features and components included in the training pipeline

than 1% of the model’s total—thereby significantly reducing both computational and storage overhead. Its modular nature allows adapters to be added or removed without altering the base model weights, facilitating plug-and-play experimentation across different configurations.

```
lora_config = LoraConfig(
    r=4,
    lora_alpha=8,
    lora_dropout=0.0,
    target_modules=["q_proj", "v_proj", "o_proj"],
    bias="none",
    task_type="CAUSAL_LM",
)
```

Listing 4.3: LoRA configuration for causal language modeling using PEFT

Moreover, LoRA adapters support better generalization by mitigating catastrophic forgetting, preserving the base model’s pre-trained knowledge while enabling effective learning of task-specific objectives. This configuration is implemented using Hugging Face’s `peft`³ library, which provides seamless integration of LoRA within transformer-based models. The configuration details of the LoRA adapter modules are listed in Table 4.3.

4.3.3 Training Pipeline Setup

This training pipeline uses Hugging Face’s `transformers`⁴, `accelerate`⁵, and `datasets`⁶ frameworks. The tokenized dataset is streamed into the model in batched form with support for gradient accumulation, mixed precision, and distributed training.

This training loop supports both standalone LLM fine-tuning and adapter-based modular fine-tuning and can be reused across multiple datasets or model architectures with minimal reconfiguration. The training configurations are listed in Table 4.4.

4.3.4 Text Generation Settings

This subsection details the configuration and hyperparameters applied during the text generation phase, which critically affect the quality, diversity, and coherence of the generated quests. The generation parameters were carefully chosen to balance creativity with

³<https://github.com/huggingface/peft>

⁴<https://github.com/huggingface/transformers>

⁵<https://github.com/huggingface/accelerate>

⁶<https://github.com/huggingface/datasets>

Parameter	Description
Prompt structure	XML-like tags (e.g., < begin_objective >)
Max sequence length	512 tokens
Temperature	0.7
Top-k sampling	50
Top-p sampling	0.9
Repetition penalty	1.2

Table 4.5: Settings used for text generation tasks

structural fidelity, considering the specific characteristics of each model architecture, such as GPT-2 and its fine-tuned variants.

These parameters listed in Table 4.5 were designed to generate high-quality, diverse quest narratives while maintaining structural consistency, ensuring that the generated outputs could be effectively evaluated using both automated metrics and human assessments.

4.4 Evaluation Strategy

To assess the quality of generated quests, a combination of quantitative and qualitative evaluation methods is employed. These include standard language modeling metrics like perplexity, reference-based n-gram overlap metrics such as BLEU and METEOR, semantic similarity scoring through BERTScore, and a qualitative analysis of sample outputs to assess structural and narrative coherence. Each metric offers complementary insight into different aspects of the generation quality: fluency, relevance, semantic fidelity, and task-specific performance.

4.4.1 Perplexity

Perplexity⁷ is a widely used intrinsic metric for evaluating language models. It quantifies how well a probability model predicts a given sequence. Formally, the perplexity of a language model M on a sequence of tokens $w_1, w_2, \dots, w_N$ is defined as:

$$\text{Perplexity}(M) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_{<i}) \right) \quad (4.1)$$

Lower perplexity indicates that the model assigns higher probability to the actual next tokens in the sequence, suggesting better fluency and alignment with the target distribution.

4.4.2 BLEU

The Bilingual Evaluation Understudy⁸ (BLEU) score is a reference-based metric originally developed for machine translation. It measures the n-gram overlap between the generated output and one or more reference texts. The BLEU score is calculated using precision

⁷<https://huggingface.co/spaces/evaluate-metric/perplexity>

⁸<https://huggingface.co/spaces/evaluate-metric/bleu>

over n-grams and includes a brevity penalty to penalize overly short outputs. For up to 4-grams, the score is given by:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^4 w_n \log p_n \right) \quad (4.2)$$

where p_n is the modified precision for n-grams, w_n is the weight (typically $\frac{1}{4}$), and BP is the brevity penalty, defined as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp \left(1 - \frac{r}{c} \right) & \text{if } c \leq r \end{cases} \quad (4.3)$$

where c is the length of the candidate and r is the length of the reference.

4.4.3 METEOR

METEOR⁹ (Metric for Evaluation of Translation with Explicit ORdering) is another reference-based metric that improves upon BLEU by incorporating stemming, synonymy matching, and flexible alignment. It computes a harmonic mean of unigram precision and recall, with a higher weight on recall. It also applies a fragmentation penalty to discourage disordered alignments. The overall score is:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Penalty}) \quad (4.4)$$

and

$$F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P} \quad (4.5)$$

where P and R are unigram precision and recall, respectively.

4.4.4 BERTScore

BERTScore¹⁰ evaluates semantic similarity by leveraging contextual embeddings from pre-trained BERT-like models. Instead of relying on exact n-gram overlap, it compares each token in the candidate with the most similar token in the reference using cosine similarity. The final score is a combination of precision, recall, and F1 computed over these similarity scores:

$$\text{BERTScore}_{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (4.6)$$

where precision P and recall R are calculated using cosine similarity between contextualized embeddings of matched tokens.

4.4.5 Qualitative Analysis

In addition to automated evaluation, a human-based qualitative analysis is conducted on a representative subset of the generated quests. This assessment focuses on several critical

⁹<https://huggingface.co/spaces/evaluate-metric/meteor>

¹⁰<https://huggingface.co/spaces/evaluate-metric/bertscore>

Criterion	Description
Narrative Coherence	Logical progression and internal consistency of the quest storyline
Goal Clarity	Clear articulation of objectives and task sequences
World Consistency	Adherence to the lore and world-specific constraints of the game
Language Fluency	Correct grammar, phrasing, and stylistic naturalness

Table 4.6: Qualitative criteria used for evaluating generated quests

dimensions: narrative coherence, which examines the logical flow and internal consistency of the storyline; goal clarity, evaluating whether the objective and associated tasks are clearly articulated; world consistency, which ensures alignment with the fictional game’s lore and setting; and language fluency, referring to grammatical correctness and natural phrasing. These qualitative insights help contextualize the numerical results and capture nuanced attributes of quest quality that are not fully measurable by automated metrics. The various qualitative criteria used for evaluation are listed in Table 4.6.

Chapter 5

Results and Discussion

In this chapter, we present the outcomes of our experiments, encompassing both quantitative evaluations and comparative analyses across different model configurations. The aim is to assess the effectiveness of PQG using language models fine-tuned with parameter-efficient techniques such as LoRA and quantized via the NF4 format. Quantitative metrics—namely Perplexity, BLEU, METEOR, and BERTScore—are employed to evaluate both the linguistic fluency and semantic accuracy of the generated quests. These metrics are reported across baseline and fine-tuned versions of selected models to capture improvements attributable to our training pipeline.

Beyond numerical evaluations, we also explore model behavior through comparative analysis to highlight strengths, weaknesses, and trade-offs between performance and efficiency. This includes assessing how quantization impacts output quality and whether LoRA-based adaptation leads to better task-specific generalization.

Finally, we provide an interpretative discussion contextualizing these results within the broader scope of the project, including reflections on the utility of automated metrics, qualitative patterns observed in generated samples, and practical implications for future applications in game development.

5.1 Training Results

The training phase across all models was conducted over a single epoch, with progressive learning rate schedules and full logging of loss and gradient metrics. This section outlines the training behavior and convergence characteristics observed during fine-tuning.

All models demonstrated a clear downward trend in training loss, reflecting consistent optimization progress. GPT-2 Large showed the steepest decline, ending with a final training loss of 1.6781, while TinyLLaMA-1.1B-Chat achieved the lowest overall value at 1.3555. In contrast, the base GPT-2 model plateaued around 2.4868, suggesting underfitting relative to the other models.

Evaluation loss values followed a similar trend, with TinyLLaMA again achieving the best result at 1.268, followed by GPT-2 Large at 1.523 and GPT-2 Medium at 1.882. These results affirm that lower training loss generally correlated with stronger generalization to the held-out evaluation data.

Gradient norms provide insight into training stability. GPT-2 and GPT-2 Medium maintained stable and relatively low gradients (around 0.3-0.6), indicating smooth learning curves. In contrast, TinyLLaMA exhibited initially high gradient norms (up to 2.5)

Model	Final Train Loss	Eval Loss	Max Grad Norm	Min Grad Norm
GPT-2	2.4868	2.1573	0.6707	0.2780
GPT-2 Medium	2.1188	1.8826	0.4840	0.2307
GPT-2 Large	1.6781	1.5232	0.4557	0.2274
LLaMA-3.2-1B-Instruct	1.8126	1.6759	1.9814	1.0357
TinyLLaMA-1.1B-Chat-v1.0	1.3555	1.2684	2.5458	1.0018

Table 5.1: Training and evaluation metrics reported for each fine-tuned model

Model	Perplexity	BLEU	METEOR	F1
GPT-2	8.65	0.0221	0.2414	0.7883
GPT-2 Medium	6.57	0.0242	0.2544	0.7921
GPT-2 Large	4.59	0.0266	0.2297	0.7862
LLaMA-3.2-1B-Instruct	5.34	0.0256	0.2377	0.7857
TinyLLaMA-1.1B-Chat-v1.0	3.56	0.0307	0.2184	0.7879

Table 5.2: Detailed quantitative evaluation results for each fine-tuned model

which steadily decreased, suggesting rapid adaptation followed by convergence. This steep early learning curve likely contributed to its superior generalization performance.

The learning rate schedule, linearly increasing from $3\text{e-}6$ to $3\text{e-}5$, facilitated effective warm-up across all models. No instabilities such as gradient explosions or sudden loss spikes were observed, confirming the suitability of the chosen training configuration for single-epoch fine-tuning.

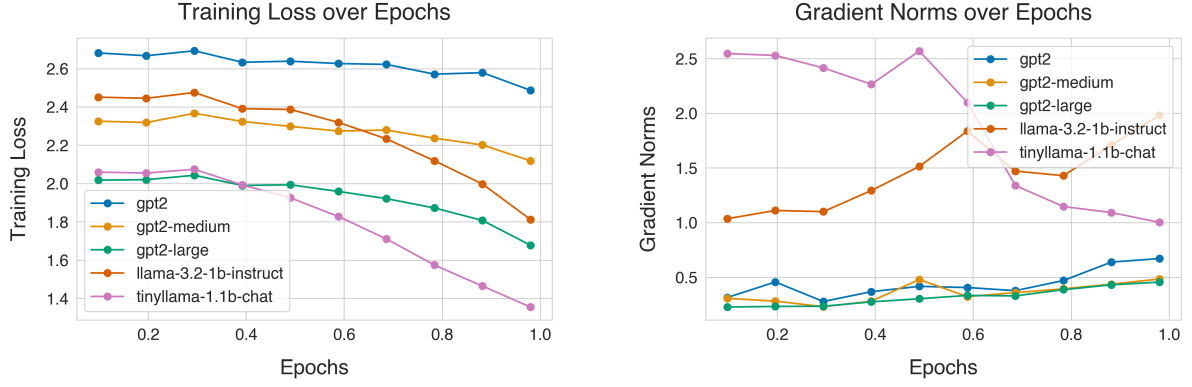
In general, the training performance aligns with the evaluation results: models that trained more efficiently and achieved lower losses also performed better on generation metrics. Additionally, the findings suggest that smaller, modern architectures such as TinyLLaMA can utilize compact parameterization for efficient fine-tuning while maintaining high output quality.

5.2 Quantitative Evaluation Results

Building on the hypothesis from related work [25], which suggests that the GPT-2 and the LLaMA pretraining dataset lacks a sufficient number of quest examples for high-quality quest generation, we aimed to address this limitation through fine-tuning on a specialized dataset.

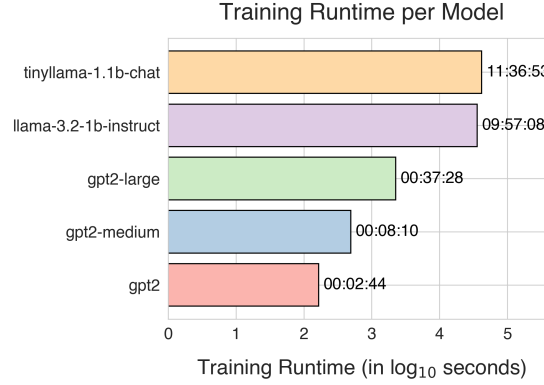
Following the completion of fine-tuning and text generation, we evaluated model performance using a suite of automatic metrics: Perplexity, BLEU, METEOR, and BERTScore. These were chosen to capture various dimensions of output quality, such as fluency, lexical overlap with reference texts, and semantic similarity. To ensure consistency across model comparisons, all generations were produced using identical text generation settings, including prompt structure, sampling strategies, and maximum sequence lengths, as described in Section 4.3.4.

By applying these metrics to both GPT-2 and LLaMA models with their increasing sizes, we aim to quantify the improvements attributable to domain-specific learning. Moreover, these quantitative evaluations form the basis for further model comparisons, allowing us to investigate how architecture type, parameter scale, and fine-tuning strategy



(a) Training loss over epochs. Lower final loss indicates better convergence; sharp early declines reflect faster initial learning.

(b) Gradient norm values. Stable norms imply smooth optimization; spikes may reflect instability or poor learning rate settings.



(c) Training runtime (log scale). Bars show log-scaled durations in seconds; labels denote exact times per model.

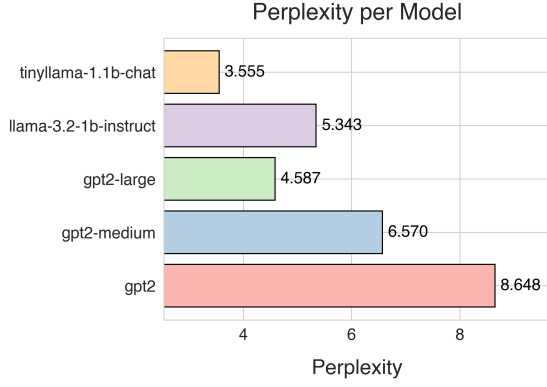
Figure 5.1: Training diagnostics for fine-tuned models. The plots display loss progression, gradient norm stability, and overall training time, offering insight into optimization dynamics and efficiency.

contribute to performance in structured narrative generation tasks. Results are reported in Table 5.2 and are further analyzed in the following sections.

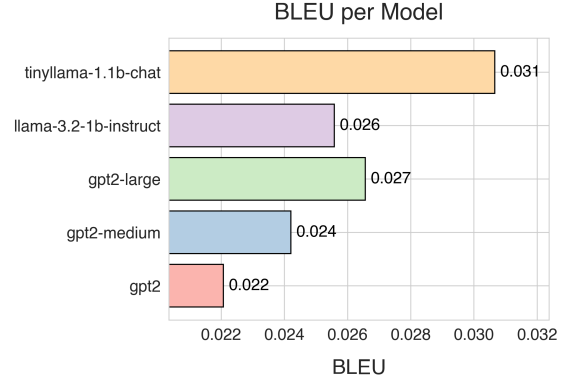
The quantitative evaluation results offer clear insights into the effectiveness of fine-tuning for quest generation. Perplexity, a measure of language model confidence, was lowest for TinyLLaMA 1.1B Chat (3.56), indicating its strong capability in producing coherent, fluent outputs. This suggests that smaller instruction-tuned models, when fine-tuned properly, can generalize effectively to domain-specific tasks.

In terms of BLEU, which assesses n-gram overlap with reference text, TinyLLaMA again led with a score of 0.0307. This reflects its strength in mimicking the syntactic structure of expected quest outputs. However, GPT-2 Large and LLaMA 3.2-1B also performed competitively, indicating that larger models retain reasonable generalization post fine-tuning.

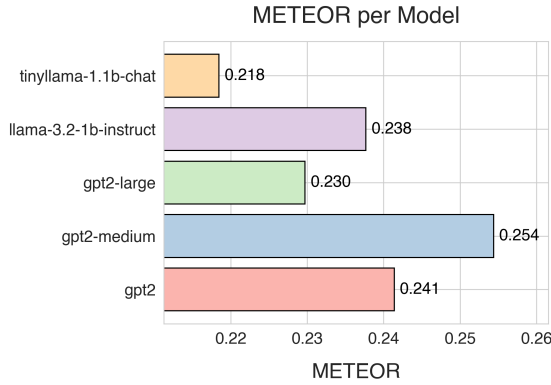
Interestingly, GPT-2 Medium achieved the highest METEOR score (0.2544), which evaluates semantic alignment and linguistic variation. This suggests that it captures the



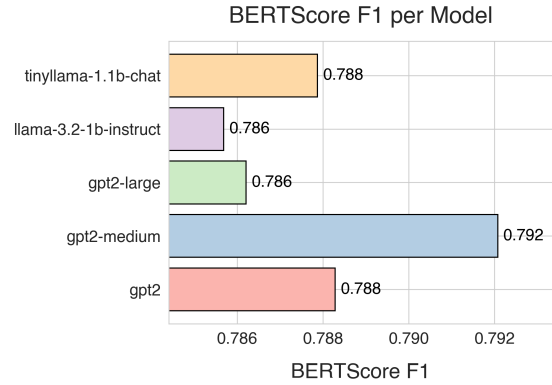
(a) Perplexity scores after fine-tuning. Lower values indicate better language modeling ability.



(b) BLEU scores of generated quests. Higher scores reflect stronger lexical overlap with references.



(c) METEOR scores, incorporating synonymy and stemming. Higher values indicate better semantic alignment.



(d) BERTScore F1 scores. Measures contextual semantic similarity using transformer embeddings.

Figure 5.2: Automated evaluation results across models. The plots illustrate language modeling quality (Perplexity) and output similarity with reference quests using BLEU, METEOR, and BERTScore.

meaning and phrasing of quests more effectively than other models. Meanwhile, TinyLLaMA achieved the best BERTScore F1 (0.7879), indicating strong contextual similarity between generated and reference quests.

These results validate our hypothesis that pretraining alone is insufficient for high-quality quest generation. Fine-tuning on a structured dataset significantly boosts performance across metrics. Notably, each model exhibits different strengths—TinyLLaMA in fluency and structure, GPT-2 Medium in semantic coherence—implying that model selection should align with specific generation goals.

5.3 Discussion

The results of this study demonstrate that fine-tuning LLMs on structured, domain-specific quest data yields substantial improvements in both generation quality and model

Category	Värtinen et al. (2024) [24]	This Study (2025)
Model & Input Format	Fine-tuned GPT-2 on structured XML-like quest data with predefined fields like <code>< begin_quest.giver ></code> and <code>< begin_objective ></code>	Fine-tuned GPT-2, TinyLLaMA, and LLaMA 3.2 models using similarly tagged XML-style quest data aligned for prompt-response conditioning
Prompting & Generation	Used prompt-based decoding with metadata tags to guide generation and compared various metadata formats for their effectiveness (narrative, simple, XML-like)	Extended the prompt format with additional control tags and instruction-based formatting for richer generation control on modern chat-oriented large models
Quantitative Evaluation	Reported conditional model perplexity of 11.05 for Quest-GPT-2 after fine-tuning on their quest dataset (raw XML-like format)	Achieved lower perplexity: 3.56 for TinyLLaMA and 4.59 for GPT-2 Large, suggesting better token prediction confidence
Qualitative Evaluation	Survey-based human evaluation showed preference for human-authored content but recognized the structured model output as plausible and creative	Used automatic metrics (BLEU, METEOR, BERTScore) applied in various NLG tasks to compare the semantic coherence of the text
Evaluation Insights	Evaluation underscored the importance of structured input in improving generation quality and included a human-subject study to assess the plausibility and relevance of the generated quests	Reinforced structured prompts and LoRA-based fine-tuning as effective for generating coherent, game-aligned quests in resource-constrained settings

Table 5.3: Comparison of quest generation strategies and outcomes between Värtinen et al. [24] and this study.

convergence. Across evaluation and training metrics, clear patterns emerge that highlight the effectiveness of targeted adaptation for PCG in RPGs.

From an evaluation standpoint, models such as TinyLLaMA-1.1B-Chat, GPT-2 Large, and GPT-2 Medium consistently achieved higher scores across most metrics. TinyLLaMA stood out with the best performance on BLEU, ROUGE, and BERTScore, indicating strong lexical overlap and semantic similarity with ground truth references. While GPT-2 Medium showed slightly lower lexical overlap in BLEU, it achieved the highest METEOR score, suggesting better handling of synonymy and paraphrasing. These results support the hypothesis that larger models or more modern architectures can better model the structured linguistic features of quests, especially when fine-tuned on relevant data.

Training dynamics also reflect this trend. All models demonstrated decreasing training and evaluation losses across the single-epoch run, with TinyLLaMA achieving the lowest evaluation loss (1.268), followed by GPT-2 Large (1.523) and LLaMA-3.2-1B-Instruct (1.676). This loss reduction was accompanied by stable learning rate schedules and manageable gradient norms. While TinyLLaMA initially exhibited high gradient norms, these values decreased over time, indicating effective adaptation. In contrast, GPT-2 variants displayed smooth and stable training trajectories with consistent learning progress.

The alignment between evaluation metrics and training loss suggests that models with lower validation loss typically produced more semantically accurate and fluent quests. Importantly, the high performance of TinyLLaMA—despite its relatively small parameter count—demonstrates that smaller models, when fine-tuned effectively, can compete with or surpass larger ones on domain-specific tasks. This highlights their potential for use in real-time or resource-constrained game development environments.

Compared to the experiments conducted by Värtinen et al. [24], which reported issues such as overfitting, inconsistent handling of entities, and incomplete coverage of quest elements, the models evaluated in this study exhibited more stable training behav-

ior and stronger alignment with target outputs. While the earlier work noted that the larger GPT-2 variant outperformed its smaller counterpart in coherence and coverage, this study finds similar trends, with larger or more recent models like TinyLLaMA and GPT-2 Large achieving higher scores across multiple evaluation metrics. Additionally, although the reference study observed higher perplexity with XML-like formats, the prompt-aligned structure used here did not appear to negatively impact model performance. These findings suggest that model architecture, fine-tuning configuration, and prompt design all play important roles in shaping the quality and consistency of generated quests, particularly in domain-specific contexts. A detailed comparison of the approaches and results from both studies is provided in Table 5.3.

In summary, the experiments validate the role of fine-tuning in enhancing LLM performance on PQG. Both quantitative metrics and training behavior confirm that adaptation enables models to better internalize structural patterns, narrative conventions, and content constraints of the RPG domain. These findings lay a strong foundation for further research involving longer training schedules, multi-turn dialogue integration, or hybrid systems incorporating symbolic planners and knowledge graphs [23].

Chapter 6

Conclusion and Future Work

In this chapter, the major contributions, findings, and limitations of this thesis are summarized. The chapter also outlines directions for future research and practical extensions of the proposed methods for PQG using LLMs. The goal is to consolidate the outcomes derived from the training and evaluation phases and to provide a critical outlook on how these findings inform future work in PCG for RPGs.

6.1 Summary of Contributions

In this work, we focused on exploring and evaluating various state-of-the-art language models for PQG in RPGs. Our contributions can be summarized in several key aspects.

Firstly, we conducted a comprehensive evaluation of several LLMs, including GPT-2, LLaMA-3, and TinyLLaMA. These models were fine-tuned and tested on tasks such as quest generation and performance benchmarking. Our evaluations included both training and generation metrics like BLEU, METEOR, BERTScore, and perplexity. These results provide valuable insights into how these models perform in the specific context of text generation for RPG quests and reveal the advantages and limitations of smaller, efficient models like TinyLLaMA for fine-tuning tasks. Through this, we were able to determine that TinyLLaMA, despite being a smaller model, offered significant benefits in terms of both efficiency and output quality when fine-tuned.

Secondly, we analyzed the impact of fine-tuning on the models. We carefully examined training loss, gradient norms, and perplexity during the fine-tuning process, revealing how these factors influenced model performance. This analysis showed that smaller models, although limited in size, can outperform larger models in terms of the quality of their outputs, provided that they are fine-tuned effectively. The TinyLLaMA model, in particular, demonstrated that smaller models can strike an efficient balance between computational cost and generation quality.

Another significant contribution was the development of a PQG framework that integrates text generation models into the quest design process. The system enables the dynamic creation of diverse and engaging quests by leveraging the strengths of the evaluated models, generating quest descriptions, objectives, tasks, and dialogues that are both coherent and contextually appropriate.

We also developed a robust evaluation framework to assess the generated content quantitatively and qualitatively. This framework included both automated text metrics (such as BLEU and METEOR) and human evaluation based on creativity, coherence, and completeness. Additionally, we incorporated perplexity as a measure of the fluency of

generated text. These evaluations helped us to understand the strengths and weaknesses of the models, leading to practical recommendations for generating high-quality quests.

Overall, the research contributes to the academic understanding of LLM performance in RPG quest generation and provides a practical system for dynamic quest creation. The insights gained from this work extend the potential applications of AI in interactive gaming experiences.

6.2 Limitations of the Study

Despite the promising results and the valuable insights gained from this study, several important limitations must be acknowledged to present a balanced view of the research. These limitations span the areas of model architecture, dataset coverage, evaluation methodology, and real-world applicability, each of which presents challenges that could affect the generalizability, stability, or scalability of the findings. Recognizing these constraints is essential not only for interpreting the results accurately but also for guiding future efforts aimed at improving PQG systems.

Firstly, the models used in this research, particularly GPT-2 and TinyLLaMA, while state-of-the-art in certain contexts, remain constrained by the quality and scope of their pretraining data. For example, GPT-2’s pretraining corpus lacked sufficient quest-specific examples, which limited its ability to generate high-quality, diverse quests. While TinyLLaMA demonstrated high efficiency, it exhibited some instability during fine-tuning, which can hinder its practical deployment for large-scale or real-time applications.

Secondly, the dataset used in this research, which was derived from older RPG quests, may not fully represent the diversity and evolving trends in modern RPG quest design. This limitation suggests that the models’ generalization ability might be restricted when applied to newer games or more varied quest types. A more diverse and updated dataset could help address this issue, offering broader applicability for the models in modern game design.

Another limitation is the reliance on both automated metrics and human judgment for evaluating the quality of generated quests. Automated metrics like BLEU and METEOR can fail to capture the creativity, depth, and nuance of narrative content, which are critical aspects of quest generation. Additionally, human evaluators may bring their own biases, which could lead to inconsistencies in the evaluation process.

The scalability of these models also presents a challenge. While the models were evaluated on smaller-scale tasks, real-time quest generation in full-scale games presents additional computational and integration challenges. Using large language models for real-time generation could result in high latency and computational costs, especially in dynamic game environments with a large number of players.

Finally, while the focus of this research was on RPG quest generation, the models’ generalization to other types of narrative generation or interactive storytelling remains untested. Expanding the evaluation to other genres of content could provide a more comprehensive understanding of their capabilities.

6.3 Suggestions for Future Work

Several avenues for future work could build on the findings and address the limitations of this study, opening up broader possibilities for advancing PQG using language mod-

els. These include exploring larger and more diverse datasets curated specifically for RPG quest design, refining fine-tuning techniques using domain-adaptive or reinforcement learning strategies, and improving the structural coherence of generated quests through hybrid systems that integrate symbolic planning or knowledge graphs with language models [23]. Further research could also evaluate generated content through human-centered studies involving player engagement, narrative quality, and gameplay integration, ensuring that model outputs align with player expectations in actual game environments.

First, exploring more advanced models, such as GPT-3 and GPT-4, could yield better performance in quest generation tasks. These models are trained on larger, more diverse datasets and could potentially outperform GPT-2 in generating more coherent and varied quest content. Further experiments with models that have been fine-tuned on larger corpora of RPG-specific data could help improve the quality of generated quests.

Second, incorporating reinforcement learning (RL) into the quest generation process could enhance the quality and diversity of generated quests. By using RL techniques, models could be further fine-tuned to optimize quest structures based on specific game mechanics or player behavior. This approach could enable dynamic quest generation that adapts to a player’s actions, making the experience more engaging and personalized.

Another promising direction is the development of real-time quest generation systems. Future research could focus on reducing the computational load required by large models for real-time content generation. This could be achieved by optimizing model architectures, implementing model distillation, or using hybrid models that balance the strengths of larger models with the efficiency of smaller ones. Real-time quest generation could open new possibilities for dynamic storytelling in video games, where quests evolve in response to player decisions and interactions.

Expanding the dataset used in this research is another important step for future work. A more extensive and updated dataset that includes a diverse range of quest types from various RPGs would help the models generate a broader array of quests. This would ensure that the models are not limited by the biases inherent in older datasets and could provide more generalized results across different game genres.

Finally, further improving the evaluation framework is crucial for better assessing the quality of generated content. Incorporating player feedback, engagement metrics, and long-term player experiences could offer a more holistic view of how AI-generated quests impact gameplay. These metrics could complement existing evaluation tools and provide richer insights into the effectiveness of PQG in enhancing player immersion and engagement.

In addition to these broader research directions, several project-specific enhancements could significantly improve the current implementation. One key improvement involves supporting persistent dialogue or narrative memory, which would allow generated quests to maintain coherence and continuity across multiple interactions or quest stages. Additionally, adopting more sophisticated prompt engineering strategies could lead to more structured and contextually rich outputs, reducing dependency on rigid templates and increasing model adaptability to diverse quest formats.

On the infrastructure side, access to a larger-memory GPU would enable training larger, more capable models such as GPT-3-sized architectures or newer instruction-tuned variants. This would also make it feasible to extend training across multiple epochs and utilize mixed precision (FP16) training for improved speed and memory efficiency. Longer training schedules, combined with gradient accumulation and learning rate warm-up techniques, could lead to better convergence and more stable fine-tuning, especially

for instruction-following models adapted to the RPG domain.

To support these improvements, enhancements in the dataset pipeline are also necessary. Improved data annotation tools would facilitate efficient and consistent expansion of the training set while preserving structural integrity and narrative quality. Additionally, implementing smarter checkpointing mechanisms and adaptive learning rate schedules could reduce training instabilities, that is an issue observed particularly with lightweight models like TinyLLaMA. Finally, incorporating automated consistency rules to detect missing data and contradictions in generated outputs would further ensure the reliability and applicability of the system in real-world game development workflows.

In conclusion, expanding the scope of this research to include more advanced models, real-time generation systems, and improved evaluation techniques would further enhance the potential of AI for PCG in video games, paving the way for more dynamic, engaging, and player-driven game experiences.

6.4 Conclusion

This study demonstrated the feasibility of PQG using quantized LLMs and PEFT methods within resource-constrained environments. By developing a modular PQG pipeline, from dataset construction and prompt formatting to quantization, fine-tuning, and evaluation, the project established a foundational framework for generating narratively coherent, context-aware RPG quests. Experimental results highlighted the effectiveness of compact models like TinyLLaMA in balancing generation quality with computational efficiency, offering a practical solution for AI-assisted narrative design. Despite certain limitations related to dataset diversity, model scale, and real-time applicability, the outcomes contribute valuable insights into the integration of LLMs into PCG workflows. This work sets the stage for future research into hybrid generation strategies, reinforcement learning-driven personalization, and real-time deployment scenarios, paving the way for scalable and dynamic quest generation systems in modern game development.

Bibliography

- [1] Epyx and AI Design, *Rogue*, Game [PC], 1980.
- [2] Wizards of the Coast, *Dungeons & Dragons*, Game [Tabletop], 1974.
- [3] Mojang Studios, *Minecraft*, Game [PC], Stockholm, Sweden, 2011.
- [4] Latitude, *AI Dungeon*, Game [PC], 2019.
- [5] A. Vaswani *et al.*, “Attention is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] T. Brown *et al.*, “Language Models Are Few-shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [7] S. Mangrulkar *et al.* “PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods.” (2022), [Online]. Available: <https://github.com/huggingface/peft> (visited on 05/16/2025).
- [8] E. J. Hu *et al.*, “LoRA: Low-rank Adaptation of Large Language Models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [9] T. Dettmers *et al.*, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 10 088–10 115, 2023.
- [10] Interplay Entertainment, *Fallout*, Game [PC], Los Angeles, US, 1997.
- [11] Interplay Entertainment, *Fallout 2*, Game [PC], Los Angeles, US, 1998.
- [12] BioWare, *Baldur’s Gate*, Game [PC], Los Angeles, CA, USA, 1998.
- [13] BioWare, *Baldur’s Gate II: Shadows of Amn*, Game [PC], Los Angeles, CA, USA, 2000.
- [14] Bethesda Game Studios, *The Elder Scrolls IV: Oblivion*, Game [PC], Rockville, Maryland, United States, 2006.
- [15] Bethesda Game Studios, *The Elder Scrolls V: Skyrim*, Game [PC], Rockville, Maryland, United States, 2011.
- [16] J. Togelius *et al.*, “Procedural Content Generation: Goals, Challenges and Actionable Steps,” in *Artificial and Computational Intelligence in Games*, ser. Dagstuhl Follow-Ups, S. M. Lucas *et al.*, Eds., vol. 6, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2013, pp. 61–75, ISBN: 978-3-939897-62-0. DOI: 10.4230/DFU.Vol6.12191.61.
- [17] J. B. Black and R. Wilensky, “An Evaluation of Story Grammars,” *Cognitive Science*, vol. 3, no. 3, pp. 213–229, 1979.
- [18] M. O. Riedl and V. Bulitko, “Interactive Narrative: An Intelligent Systems Approach,” *AI Magazine*, vol. 34, no. 1, pp. 67–67, 2013.

- [19] M. Hendrikx *et al.*, “Procedural Content Generation for Games: A Survey,” *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 9, no. 1, pp. 1–22, 2013.
- [20] H. Touvron *et al.*, “Llama: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [21] R. Bommasani *et al.*, “On the Opportunities and Risks of Foundation Models,” *arXiv preprint arXiv:2108.07258*, 2021. [Online]. Available: <https://arxiv.org/pdf/2108.07258> (visited on 05/16/2025).
- [22] P. Ammanabrolu *et al.*, “Toward Automated Quest Generation in Text-Adventure Games,” *arXiv preprint arXiv:1909.06283*, 2019.
- [23] T. Ashby *et al.*, “Personalized Quest and Dialogue Generation in Role-playing Games: A Knowledge Graph-and Language Model-based Approach,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–20.
- [24] S. Värtinen *et al.*, “Generating Role-Playing Game Quests With GPT Language Models,” *IEEE Transactions on Games*, vol. 16, no. 1, pp. 127–139, 2024. DOI: 10.1109/TG.2022.3228480.
- [25] J. van Stegeren and J. Myśliwiec, “Fine-tuning GPT-2 on Annotated RPG Quests for NPC Dialogue Generation,” in *Proceedings of the 16th International Conference on the Foundations of Digital Games*, 2021, pp. 1–8.
- [26] Y. Luo *et al.*, “An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning,” *arXiv preprint arXiv:2308.08747*, 2023. [Online]. Available: <https://arxiv.org/pdf/2308.08747> (visited on 05/16/2025).
- [27] L. Huang *et al.*, “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [28] K. Papineni *et al.*, “BLEU: A Method for Automatic Evaluation of Machine Translation,” 2002, pp. 311–318.
- [29] S. Banerjee and A. Lavie, “METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments,” in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, Ann Arbor, Michigan: Association for Computational Linguistics, Jun. 2005, pp. 65–72. [Online]. Available: <https://www.aclweb.org/anthology/W05-0909> (visited on 05/16/2025).
- [30] A. Celikyilmaz *et al.*, “Evaluation of Text Generation: A Survey,” *arXiv preprint arXiv:2006.14799*, 2020. [Online]. Available: <https://arxiv.org/pdf/2006.14799> (visited on 05/16/2025).
- [31] T. Zhang *et al.*, “BERTScore: Evaluating Text Generation with BERT,” *arXiv preprint arXiv:1904.09675*, 2019. [Online]. Available: <https://arxiv.org/pdf/1904.09675> (visited on 05/16/2025).
- [32] A. Radford *et al.*, “Language Models Are Unsupervised Multitask Learners,” *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [33] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” 2023.

- [34] P. Zhang *et al.*, *TinyLlama: An Open-Source Small Language Model*, 2024. arXiv: 2401.02385 [cs.CL]. [Online]. Available: <https://github.com/jzhang38/TinyLlama> (visited on 05/16/2025).
- [35] Lightning AI. “Lit-gpt.” (2023), [Online]. Available: <https://www.github.com/Lightning-AI/lit-gpt> (visited on 05/16/2025).
- [36] N. Houlsby *et al.*, “Parameter-efficient Transfer Learning for NLP,” in *International Conference on Machine Learning*, PMLR, 2019, pp. 2790–2799.
- [37] A. Radford *et al.*, “Improving Language Understanding by Generative Pre-training,” 2018.
- [38] Y. Shibata *et al.*, “Byte Pair Encoding: A Text Compression Scheme that Accelerates Pattern Matching,” 1999.
- [39] X. Song *et al.*, “Fast WordPiece Tokenization,” *arXiv preprint arXiv:2012.15524*, 2020. [Online]. Available: <https://arxiv.org/pdf/2012.15524> (visited on 05/16/2025).
- [40] T. Kudo and J. Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,” *arXiv preprint arXiv:1808.06226*, 2018. [Online]. Available: <https://arxiv.org/pdf/1808.06226> (visited on 05/16/2025).
- [41] J. Howard, *Quests: Design, Theory, and History in Games and Narratives*. AK Peters/CRC Press, 2022.
- [42] The Elder Scrolls Wiki. “Radiant Quests.” (2011), [Online]. Available: https://elderscrolls.fandom.com/wiki/Radiant_Quests (visited on 05/16/2025).
- [43] R. E. Fikes and N. J. Nilsson, “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving,” *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [44] D. Thue *et al.*, “Interactive Storytelling: A Player Modelling Approach,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 3, 2007, pp. 43–48.
- [45] J. Dormans, “Adventures in Level Design: Generating Missions and Spaces for Action Adventure Games,” in *Proceedings of the 2010 Workshop on Procedural Content Generation in Games*, 2010, pp. 1–8.
- [46] J. Doran and I. Parberry, “A Prototype Quest Generator Based on a Structural Analysis of Quests from Four MMORPGs,” in *Proceedings of the 2nd International Workshop on Procedural Content Generation in Games*, 2011, pp. 1–8.
- [47] J. Togelius *et al.*, “Search-based Procedural Content Generation: A Taxonomy and Survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [48] N. Shaker *et al.*, *Procedural Content Generation in Games*. Springer Nature, 2016. DOI: 10.1007/978-3-319-42716-4.
- [49] J. Liu *et al.*, “Deep Learning for Procedural Content Generation,” *Neural Computing and Applications*, vol. 33, no. 1, pp. 19–37, 2021.
- [50] M. Si *et al.*, “THESPIAN: An Architecture for Interactive Pedagogical Drama,” in *AIED*, 2005, pp. 595–602.

- [51] C. Kerr and D. Szafron, “Supporting Dialogue Generation for Story-based Games,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 5, 2009, pp. 154–160.
- [52] V. Breault *et al.*, “Let CONAN Tell You a Story: Procedural Quest Generation,” *Entertainment Computing*, vol. 38, p. 100 422, 2021.
- [53] Blizzard Entertainment, *World of Warcraft*, Game [PC], Irvine, California, United States, 2004.
- [54] M. Guzdial *et al.*, “Visual Procedural Content Generation with An Artificial Abstract Artist,” in *Proceedings of ICCG Computational Creativity and Games Workshop*, 2017.
- [55] Y. Liang *et al.*, “Procedural Content Generation of Rhythm Games using Deep Learning Methods,” in *Entertainment Computing and Serious Games: First IFIP TC 14 Joint International Conference, ICEC-JCSG 2019, Arequipa, Peru, November 11–15, 2019, Proceedings 1*, Springer, 2019, pp. 134–145.
- [56] A. Sarkar *et al.*, “Controllable Level Blending between Games using Variational Autoencoders,” *arXiv preprint arXiv:2002.11869*, 2020.
- [57] M.-A. Côté *et al.*, “TextWorld: A Learning Environment for Text-based Games,” in *Computer Games: 7th Workshop, CGW 2018, Held in Conjunction with the 27th International Conference on Artificial Intelligence, IJCAI 2018, Stockholm, Sweden, July 13, 2018, Revised Selected Papers 7*, Springer, 2019, pp. 41–75.
- [58] A. Joshi *et al.*, “Likert Scale: Explored and Explained,” *British Journal of Applied Science & Technology*, vol. 7, no. 4, p. 396, 2015.
- [59] H. Abdi and L. J. Williams, “Tukey’s Honestly Significant Difference (HSD) Test,” *Encyclopedia of Research Design*, vol. 3, no. 1, pp. 1–5, 2010.
- [60] C.-Y. Lin and F. Och, “Looking for a Few Good Metrics: ROUGE and its Evaluation,” in *Ntcir Workshop*, 2004, pp. 1–8.
- [61] G. Smith *et al.*, “PCG-based Game Design: Enabling New Play Experiences through Procedural Content Generation,” in *Proceedings of the 2nd International Workshop on Procedural Content Generation in Games*, 2011, pp. 1–4.
- [62] Runic Games, *Torchlight II*, Game [PC], Seattle, Washington, U.S., 2012.
- [63] Nukapedia: The Fallout Wiki. “Fallout Quests.” (1997), [Online]. Available: https://fallout.fandom.com/wiki/Fallout_quests (visited on 05/16/2025).
- [64] Nukapedia: The Fallout Wiki. “Fallout 2 Quests.” (1998), [Online]. Available: https://fallout.fandom.com/wiki/Fallout_2_quests (visited on 05/16/2025).

Appendix A

Format for the Quest Dataset

This appendix presents the formal specification of the quest dataset used in this work. The dataset consists of structured representations of RPG-style quests, annotated and stored in machine-readable formats. These formats are essential for supervised fine-tuning and inference using LLMs. Two formats are used: a native JSON representation for data storage and preprocessing, and a structured prompt format in XML-like syntax for training autoregressive LLMs.

JSON Format Specification

The primary dataset is stored in JSON, where each quest is an object containing fields such as the quest giver, objectives, task list, reward structure, and other metadata. This format is optimal for parsing and preprocessing during the data pipeline. A sample entry is provided below:

```
{
  "name": "Kill Razor",
  "objective": "Eliminate Razor, the leader of the Blades.",
  "first_tasks": [
    "Locate Razor in Adytum.",
    "Engage and eliminate Razor."
  ],
  "first_task_locations": [
    {
      "name": "Adytum",
      "description": "A settlement in the Boneyard where the Blades
        reside."
    }
  ],
  "quest_giver": {
    "name": "The Regulators",
    "description": "A militia-like group controlling Adytum, enforcing
      their own brand of order.",
    "location": "Adytum"
  },
  "reward": [
    {
      "name": "Caps",
      "description": "A monetary reward for completing the task.",
      "amount": 0
    }
  ],
}
```

Field	Type	Description	Essential
name	string	Internal quest identifier, used for indexing.	✓
objective	string	Main goal or player task in the quest.	✓
first_tasks	array<string>	Step-by-step initial quest actions.	✓
first_task_locations	array<object>	Locations relevant to initial tasks.	✗
quest_giver	object	Entity assigning the quest; includes name, description, and location.	✓
reward	array<object>	Incentives for completion, e.g., items or faction favor.	✗
tools	array<string>	Tools/resources needed for completion.	✗
description	string	Narrative summary with context.	✓
items	string/array	Related items, "NONE" if unused.	✗
characters	string/array	Involved or affected NPCs.	✗
groups	string/array	Factions or organizations involved.	✗
locations	string/array	All geographical areas involved.	✗
enemies	string/array	Hostile entities in the quest.	✗
title	string	Game or universe title.	✓
motivation	string	Player's or NPC's reason to engage.	✗

Table A.1: Field description in the JSON schema proposed by Värtinen et al. [24]

```

{
  "name": "Favor with the Regulators",
  "description": "Increased reputation with the Regulators for
    carrying out their orders."
}
],
"tools": [
  "Weapons",
  "Armor"
],
"description": "The Regulators of Adytum have ordered the
  assassination of Razor, the leader of the Blades. You must locate
  and kill her.",
"items": "NONE",
"characters": "NONE",
"groups": "NONE",
"locations": "NONE",
"enemies": "NONE",
"title": "Fallout",
"motivation": "NONE"
}

```

Listing A.1: Sample Quest Dataset Format (JSON)

Field Descriptions

Each field in the JSON object plays a specific role in guiding the training, fine-tuning, or evaluation of large language models, providing structured context and supervision for generation tasks. The various field descriptions for the *Quest* dataset are given in Table A.1.

XML-like Prompt Format for Autoregressive LLMs

For training decoder-only transformer models in a CLM setting, quests are converted into a structured prompt format using XML-like tags. This format is designed to align with tokenization behavior and provide field-level structure to aid in model learning.

```
<|begin_quest|>
<|begin_objective|>
Eliminate Razor, the leader of the Blades.
<|end_objective|>
<|begin_tasks|>
Locate Razor in Adytum.
Engage and eliminate Razor.
<|end_tasks|>
<|begin_quest_giver|>
The Regulators: A militia-like group controlling Adytum, enforcing their
    own brand of order.
<|end_quest_giver|>
<|begin_task_locations|>
Adytum: A settlement in the Boneyard where the Blades reside.
<|end_task_locations|>
<|begin_rewards|>
Caps: A monetary reward for completing the task. (amount: 0)
Favor with the Regulators: Increased reputation with the Regulators for
    carrying out their orders. (amount: NONE)
<|end_rewards|>
<|end_quest|>
<|begin_description|>
The Regulators of Adytum have ordered the assassination of Razor, the
    leader of the Blades. You must locate and kill her.
<|end_description|>
```

Listing A.2: Sample Quest Dataset Format (XML-like)

Use in Training and Inference

The structured prompt aids both training (aligning fields with their values) and generation (where the model completes a partial prompt based on learned patterns).

```
{
  "prompt": "<|begin_quest|>\n<|begin_objective|>\nEliminate Razor, the
    leader of the Blades.\n<|end_objective|>\n<|begin_tasks|>\nLocate
    Razor in Adytum.\nEngage and eliminate Razor.\n<|end_tasks|>\n<|
    begin_quest_giver|>\nThe Regulators: A militia-like group
    controlling Adytum, enforcing their own brand of order.\n<|
    end_quest_giver|>\n<|begin_task_locations|>\nAdytum: A settlement
    in the Boneyard where the Blades reside.\n<|end_task_locations|>\n
    <|begin_rewards|>\nCaps: A monetary reward for completing the task.
    (amount: 0)\nFavor with the Regulators: Increased reputation with
    the Regulators for carrying out their orders. (amount: NONE)\n<|
    end_rewards|>\n<|end_quest|>\n",
  "response": "The Regulators of Adytum have ordered the assassination
    of Razor, the leader of the Blades. You must locate and kill her."
}
```

Listing A.3: Sample Prompt-Response Input Format (JSON)

Appendix B

Experimental Setup and Hyperparameters

This appendix provides a detailed overview of the experimental setup and hyperparameters used throughout the training and evaluation phases of the models in this study. It includes the hardware and software specifications, as well as the specific configurations and settings used during fine-tuning the models for quest generation tasks.

Hardware and Software Setup

To ensure the effective training and evaluation of the language models, a well-configured hardware setup and an optimized software stack were employed. The hardware configuration was selected to support the computational demands of fine-tuning large-scale transformer-based models, while the software environment was tailored for compatibility, stability, and efficiency in training workflows. The specifications for the hardware and software environments are provided in Table B.1 and Table B.2, respectively.

Models Used

The models utilized in this study include pre-trained architectures from the OpenAI and Meta AI research communities—specifically, GPT-2 and various LLaMA variants. These models were selected for their balance between accessibility, architectural diversity, and performance on text generation tasks. A summary of the models used is presented in Table B.3. Each model was initially pre-trained on large-scale corpora (e.g., the WebText dataset for GPT-2) and subsequently fine-tuned for the domain-specific task of procedural quest generation. This fine-tuning involved custom prompt formats, tailored tokenization procedures, and controlled training conditions to align model behavior with the narrative and structural conventions of RPG quests.

Preprocessing Steps

Before initiating fine-tuning, the raw quest dataset was subjected to a comprehensive preprocessing pipeline. These preprocessing steps were essential to transform the heterogeneous data into a format compatible with transformer-based language models. They

Component	Specification
GPU	NVIDIA GeForce RTX 3050 Laptop (4GB VRAM)
CPU	11th Gen Intel(R) Core i7-11370H 4-Core Processor
RAM	16GB DDR4
Storage	512GB SSD

Table B.1: Hardware specifications used for training and evaluation

Component	Specification
Operating System	Ubuntu 20.04.6 LTS
CUDA	CUDA 12.8
Python	Python 3.13.2
Docker	Docker 20.0.4
PyTorch	PyTorch 2.5.1
JupyterLab	JupyterLab 4.3.4
TensorBoard	TensorBoard 2.19.0
Hugging Face Accelerate	Accelerate 1.5.2
Hugging Face Datasets	Datasets 3.3.2
Hugging Face Evaluate	Evaluate 0.4.3
Hugging Face Transformers	Transformers 4.49.0
Docker Base Image	mambaorg/micromamba:2-cuda12.4.1-ubuntu20.04

Table B.2: Software libraries, versions, and tools used to implement the PQG system

included token normalization, metadata formatting, placeholder injection, and prompt-construction tailored to the structure of RPG quest narratives. The aim was to enhance the quality, consistency, and structural alignment of training samples, thereby improving training convergence and model performance. An overview of these preprocessing steps is provided in Table B.4.

Hyperparameters for Training

Fine-tuning hyperparameters were carefully selected to balance model convergence speed, training stability, and generalization performance. Key parameters such as batch size, learning rate, number of training steps, weight decay, and evaluation intervals were tuned based on preliminary experiments and resource availability. These parameters, detailed in Table B.5, played a central role in enabling efficient and stable learning across all models.

Hyperparameters for Generation

Following fine-tuning, the text generation phase employed a set of decoding hyperparameters to control the style, diversity, and structure of the model outputs. Parameters such as temperature, top-k and top-p sampling thresholds, repetition penalties, and maximum generation length were configured to produce coherent and contextually appropriate quest narratives. These generation hyperparameters are summarized in Table B.6, and they significantly influenced the creativity and fluency of the resulting quests.

Model	Parameters	Description
<code>gpt2[-medium,-large]</code>	124M, 355M, 774M	Each of these models was pre-trained on large-scale datasets (such as the WebText corpus for GPT-2) and then fine-tuned for quest generation tasks, with specific prompt formats and training procedures.
<code>Llama-3.2-1B-Instruct</code>	1B	Decoder-only transformer model with improved architecture, designed for more efficient performance.
<code>TinyLlama-1.1B-Chat-v1.0</code>	1.1B	A compact variant of LLaMA optimized for low-resource environments while maintaining performance.

Table B.3: List of pre-trained models used throughout the experimental setup

Step	Description
Data Collection	Gather raw quest data including objectives, tasks, and metadata.
Data Cleaning	Remove irrelevant or corrupted entries for consistency.
Text Normalization	Standardize casing, punctuation, and special characters.
Field Mapping	Assign structured fields (e.g., title, description, etc.) for training.
Special Tokens	Add tags like <code><begin_quest></code> and <code><end_quest></code> .
Dataset Splitting	Divide into train, validation, and test subsets.
Batching	Form batches to optimize memory and throughput.
Tokenization	Convert text to model-compatible token sequences.
Padding/Truncation	Fit sequences to the model’s max length.

Table B.4: Data preprocessing steps applied to the raw quest data before training

Fine-Tuning Process

The fine-tuning of pre-trained language models for PQG required careful planning to balance ideal practices with the practical constraints of available hardware. This section first describes the optimal fine-tuning methodology envisioned, followed by the actual process executed during experimentation.

Ideally, each model would be fine-tuned over multiple epochs—typically three to five—using the full training dataset. After each epoch, validation loss would be evaluated to monitor generalization and guide early stopping. Model checkpoints would be saved at every epoch, with the checkpoint having the lowest validation loss selected for final evaluation. This approach helps avoid both underfitting and overfitting, ensuring a robust trade-off between performance and generalization.

To efficiently train large models, several optimization strategies would be employed. Mixed precision training (FP16) would reduce memory use and speed up computation, gradient accumulation would simulate larger batch sizes on limited-memory GPUs, and gradient checkpointing would lower memory requirements by selectively storing intermediate activations. Ideally, a multi-GPU or distributed setup would further increase training throughput, allowing longer training runs, more extensive hyperparameter tuning, and stable convergence for large architectures like TinyLLaMA, GPT-2, or LLaMA-3 variants.

However, the actual fine-tuning process was limited by hardware constraints. Although a CUDA-enabled GPU with FP16 support was used, full mixed precision training was not possible due to compatibility issues, preventing typical gains in memory efficiency

Hyperparameter	Value	Description
Learning Rate	3e-5	Learning rate for fine-tuning the model
Batch Size	4	Batch size used for training
Epochs	1	Number of epochs for fine-tuning the models
Seed	42	Random seed for reproducibility
Gradient Accumulation	2	Number of gradient accumulation steps before updating model parameters.
Warmup Steps	100	Number of steps for the LR warm-up phase
Gradient Clipping	1.0	Gradient clipping threshold to avoid exploding gradients during training
Max Sequence Length	512	Maximum token length for input sequences
LoRA Rank	4	Controls parameter reduction for adaptation layers
LoRA Scaling Factor	8	Controls contribution of LoRA updates
LoRA Dropout Rate	0.0	Dropout rate applied to LoRA layers
Quantization Applied	4-bit	Type of quantization applied to the model

Table B.5: Hyperparameter configurations used during the fine-tuning phase

Hyperparameter	Value	Description
Prompt structure	XML-like	Structured tags (e.g., <code>< begin_objective ></code>) guide generation
Maximum sequence length	128 tokens	Caps total token count in output
Temperature	0.8	Controls randomness; higher values increase diversity
Top-k sampling	50	Samples from top 50 probable tokens per step
Top-p sampling (nucleus)	0.95	Samples tokens within 95% cumulative probability
Repetition penalty	1.2	Penalizes repeated tokens to boost diversity
Sampling enabled	True	Uses stochastic instead of deterministic decoding

Table B.6: Hyperparameter settings employed during the text generation phase

and speed. Gradient checkpointing was enabled to partially mitigate memory bottlenecks by trading computation time for reduced memory consumption, allowing the training of larger models within limited resources.

Despite these optimizations, compromises were necessary. Only 90% of the dataset was used for both training and evaluation to fit within memory limits, and each model was trained for a single epoch due to the prohibitive cost of multi-epoch training under these constraints. Validation loss was computed at the epoch’s end, and the corresponding checkpoint was saved for final evaluation.

Though this setup diverges from the ideal multi-epoch, full-dataset regime, it was sufficient for comparative analysis. The models learned meaningful task representations, and performance differences between architectures remained clear. Notably, TinyLLaMA demonstrated strong generation quality despite minimal fine-tuning, highlighting the effectiveness of modern compact models in resource-limited environments. This outcome illustrates that insightful and impactful experimentation is achievable even with constrained hardware, provided the training process is thoughtfully designed and optimized.

Evaluation Metrics

To evaluate the performance of the models both during and after fine-tuning, a comprehensive set of evaluation metrics was employed. These metrics (listed in Table B.7)

Metric	Description
Perplexity	Measures how well a language model predicts a sample. Lower values indicate better performance.
BLEU	Evaluates n-gram overlap between generated and reference texts; commonly used for machine translation.
METEOR	Considers synonymy and stem matches in addition to exact word matches, offering better alignment with human judgments.
BERTScore	Computes semantic similarity between generated and reference texts using contextual embeddings from BERT.

Table B.7: List of automated evaluation metrics and their respective roles

were chosen to capture various aspects of generation quality, including lexical accuracy, semantic relevance, fluency, and diversity. Specifically, they were used to measure the models’ effectiveness in producing coherent and contextually appropriate quest narratives—key requirements for PCG in role-playing games. The selected metrics provided a balanced assessment by combining both automated, quantitative indicators and qualitative, human-centered evaluations.